# 2024-01



数据结构　单项选择题 · 2 分

已知带头结点的非空单链表L的头指针为h，结点结构为 `[ data | next ]`，其中next是指向直接后继结点的指针。现有指针p和q，若p指向L中非首且非尾的任意一个结点，则执行下列语句序列后的结果是

```
q = p->next;
p->next = q->next;
q->next = h->next;
h->next = q;
```

A. 在p所指结点后插入q所指结点
B. 在q所指结点后插入p所指结点
C. 将p所指结点移动到L的头结点之后
D. 将q所指结点移动到L的头结点之后

王道知识与解析 · 原解析已定位

【对应知识】王道2027数据结构 · 原PDF第45页
王道2027数据结构 · 单链表插入与删除 · PDF p45

```
        p=p->next;
    return  p;                    //找到后返回该结点指针，否则返回 NULL
}
```

按值查找操作的时间复杂度为 $O(n)$。

**5. 插入结点操作**

考点追踪 ▶ 单链表插入操作的过程（2016、2024）

将值为 e 的新结点插入到第 i 个位置（成为新的第 i 个数据结点）。先检查 i 的合法性，然后找到第 i-1 个结点（前驱），再在其后插入新结点。假设找到的第 i-1 个结点为*p，然后令新结点*s 的 next 指向*p 的后继，再令*p 的 next 指向*s，如图 2.5 所示。

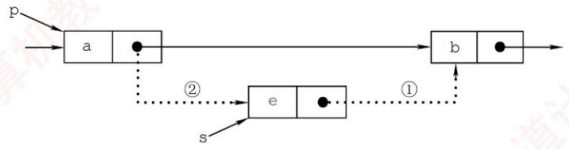



图 2.5　单链表的插入操作

【对应知识】王道2027数据结构 · 原PDF第45页（续）
王道2027数据结构 · 单链表插入与删除 · PDF p45

```
bool ListInsert(LinkList &L,int i,ElemType e){
    LNode *p=L;                   //p 指向当前扫描结点
    int j=0;                      //j 记录当前位序，头结点为第 0 个结点
    while(p!=NULL&&j<i-1){        //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p==NULL)                   //i 值不合法
        return  false;
    LNode *s=(LNode*)malloc(sizeof(LNode));
    s->data=e;
    s->next=p->next;              //图 2.5 中操作步骤①
    p->next=s;                    //图 2.5 中操作步骤②
    return  true;
}
```

**步骤①和②的顺序不可颠倒。**若先执行 p->next=s，则原后继地址丢失，导致 s->next=p->next 实际变为 s->next=s，形成自环。该操作的时间复杂度为 $O(n)$，主要开销在于查找前驱结点。若已知某结点指针，则在其后插入新结点的时间复杂度为 $O(1)$。注意，当插入位置 i=1 时，不带头结点的单链表需要更新头指针 L 指向新首结点，而带头结点的则无须特殊处理。

【对应知识】王道2027数据结构 · 原PDF第45页（续）
王道2027数据结构 · 单链表插入与删除 · PDF p45

**扩展：对某个结点进行前插操作。**

前插操作是指在某结点前插入一个新结点，与后插操作相反。任何前插操作均可转化为后插操作：只需从头结点开始顺序查找其前驱，时间复杂度为 $O(n)$。若对给定结点进行前插，则还可采用另一种技巧（适用于数据可复制的情况）：将新结点*s 插入目标结点*p 之后，再交换*p 与*s 的数据域。这种技巧逻辑上等效于前插，且时间复杂度为 $O(1)$。主要代码片段如下：

```
s->next=p->next;                    //修改指针域，不能颠倒
p->next=s;
temp=p->data;                       //交换数据域部分
p->data=s->data;
s->data=temp;
```

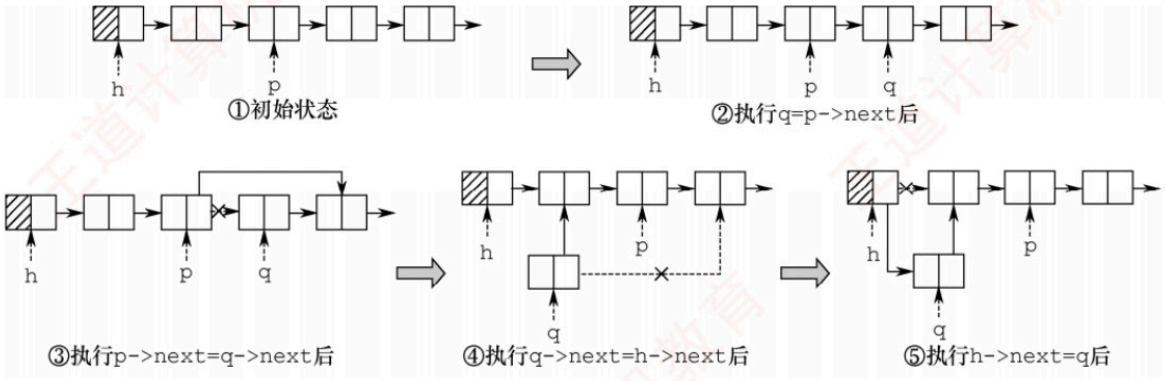
**6．删除结点操作**

删除单链表的第 i 个数据结点。先检查 i 的合法性，然后找到第 i－1 个结点（前驱），再删

【原解析】王道2027数据结构 · 原PDF第61页
王道2027数据结构 §2.3 选择题36原解析

**36．D**

假设单链表 L 的初始状态如图①所示。执行 q=p->next 后的状态如图②所示，q 指向 p 的后继结点。执行 p->next=q->next 后的状态如图③所示，结点 p 的 next 指向 q 的后继结点。执行 q->next=h->next 后的状态如图④所示，结点 q 的 next 指向 h 的后继结点。执行 h->next=q 后的状态如图⑤所示，q 所指结点移至 L 的头结点 h 之后，选项 D 正确。




对应原题本第96页；点击此行返回题目。

# 2024-02



数据结构　单项选择题 · 2 分

与表达式x+y*(z-u)/v等价的后缀表达式是

A. xyzu-*v/+
B. xyzu-v/*+
C. +x/*y-zuv
D. +x*y/-zuv

王道知识与解析 · 原解析已定位

【对应知识】3.3.2_2_栈在表达式求值中的应用(下) · 原PDF第55页
原课件 3.3.2_2_栈在表达式求值中的应用(下) · PDF p55

# 知识回顾与重要考点

用栈实现中缀表达式转后缀表达式：
初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。
从左到右处理各个元素，直到末尾。可能遇到三种情况：
① 遇到操作数。直接加入后缀表达式。
② 遇到界限符。遇到“(”直接入栈；遇到“)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。
③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。
按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

用栈实现后缀表达式的计算：
①从左往右扫描下一个元素，直到处理完所有元素
②若扫描到操作数则压入栈，并回到①；否则执行③
③若扫描到运算符，则弹出两个栈顶元素，执行相应运算，运算结果压回栈顶，回到①

用栈实现中缀表达式的计算：
初始化两个栈，操作数栈和运算符栈
若扫描到操作数，压入操作数栈
若扫描到运算符或界限符，则按照“中缀转后缀”相同的逻辑压入运算符栈（期间也会弹出运算符，每当弹出一个运算符时，就需要再弹出两个操作数栈的栈顶元素并执行相应运算，运算结果再压回操作数栈）

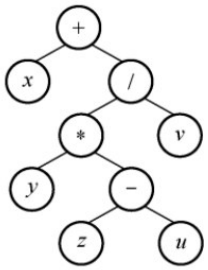
王道考研/CSKAOYAN.COM

【原解析】王道2027数据结构 · 原PDF第111页
王道2027数据结构 §3.3 选择题19原解析

**19. A**

根据中缀表达式画出对应的二叉树如右图所示，对该二叉树进行后序遍历即可得到后缀表达式为 *xyzu−*v/+*。本题也可采用本书前面描述的手算方法。

【原解析】王道2027数据结构 · 原PDF第111页
王道2027数据结构 §3.3 选择题19原解析




对应原题本第97页；点击此行返回题目。

数据结构 · 单项选择题 · 2 分

若p、q和v均为二叉树T中的结点，v有两个孩子结点，T的中序遍历序列形如：“...,p,v,q,...”，则下列叙述中，正确的是

A. p 没有右孩子，q 没有左孩子
B. p 没有右孩子，q 有左孩子
C. p 有右孩子，q 没有左孩子
D. p 有右孩子，q 有左孩子

## 王道知识与解析 · 原解析已定位

【对应知识】王道2027数据结构 · 原PDF第153页
王道2027数据结构 · 二叉树中序、后序遍历及递归实现 · PDF p153

```
void PreOrder(BiTree T){
    if(T!=NULL){
        visit(T);                  //访问根结点
        PreOrder(T->lchild);       //递归遍历左子树
        PreOrder(T->rchild);       //递归遍历右子树
    }
}
```

**2．中序遍历**（InOrder）

若二叉树为空，则不执行任何操作；否则，
1）中序遍历左子树；
2）访问根结点；
3）中序遍历右子树。

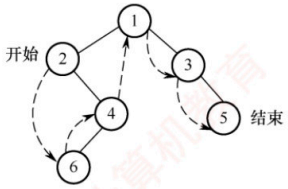



图 5.8　二叉树的中序遍历

考点追踪 ▶ **中序序列中结点关系的分析（2017、2024）**

图 5.8 中的虚线表示对该二叉树进行中序遍历的访问路径，所得中序遍历序列为 264135。

对应的递归算法如下：

```
void InOrder(BiTree T){
    if(T!=NULL){
        InOrder(T->lchild);        //递归遍历左子树
        visit(T);                  //访问根结点
        InOrder(T->rchild);        //递归遍历右子树
    }
}
```

【对应知识】王道2027数据结构 · 原PDF第153页（续）
王道2027数据结构 · 二叉树中序、后序遍历及递归实现 · PDF p153

**3．后序遍历**（PostOrder）

若二叉树为空，则不执行任何操作；否则，
1）后序遍历左子树；
2）后序遍历右子树；
3）访问根结点。

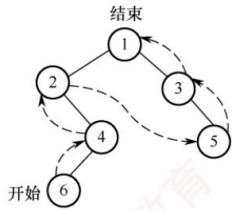



图 5.9　二叉树的后序遍历

图 5.9 中的虚线表示对该二叉树进行后序遍历的访问路径，所得后序遍历序列为 642531。

对应的递归算法如下：

```
void PostOrder(BiTree T){
    if(T!=NULL){
        PostOrder(T->lchild);      //递归遍历左子树
        PostOrder(T->rchild);      //递归遍历右子树
        visit(T);                  //访问根结点
    }
}
```

上述三种遍历算法中，递归遍历左右子树的顺序都是固定的，区别仅在于访问根结点的时机不同。不论采用哪种遍历方法，**每个结点都被访问一次且仅一次**，时间复杂度均为 $O(n)$。在递归实现中，**系统工作栈的深度等于树的高度**，在最坏情况下（如单支树），空间复杂度为 $O(n)$。

**4．层次遍历**

图 5.10 展示了二叉树的层次遍历过程，即按照 1, 2, 3, 4 的层次顺序，以及箭头所指方向，从上到下、从左到右逐层访问二叉树中的各结点。

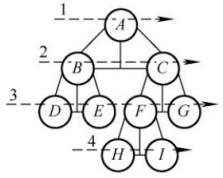
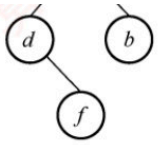



图 5.10　二叉树的层次遍历

【原解析】王道2027数据结构 · 原PDF第170页
王道2027数据结构 §5.3 选择题43原解析

**43**. A




根据中序遍历的特点，$v$ 有左右子树，以 $v$ 为根的子树的中序序列为：$v$ 的左子树的中序序列, $v$, $v$ 的右子树的中序序列。又因为 $T$ 的中序序列为“…, $p$, $v$, $q$,…”，可知 $p$ 属于 $v$ 的左子树，$q$ 属于 $v$ 的右子树。在 $v$ 的左子树的中序序列中：假设 $p$ 有右孩子，则 $p$ 的右孩子在中序序列中应在 $p$ 之后，与 $p$ 是最后一个遍历结点矛盾，因此 $p$ 不存在右孩子。在 $v$ 的右子树的中序序列中：假设 $q$ 存在左孩子，则 $q$ 的左孩子在中序序列中应在 $q$ 之前，与 $q$ 是第一个遍历结点矛盾，因此 $q$ 不存在左孩子。

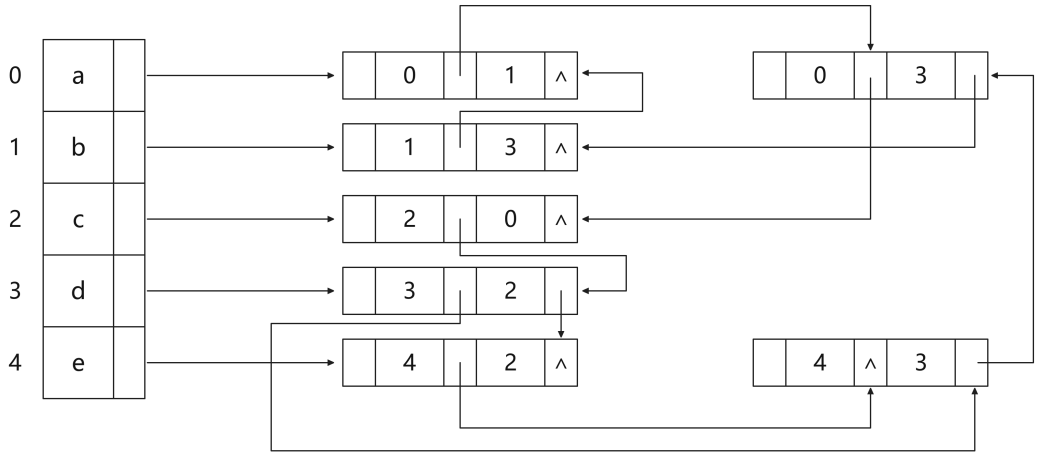
对应原题本第98页；点击此行返回题目。

# 2024-04



数据结构　单项选择题 · 2 分

若无向图 G=(V, E) 的邻接多重表如下图所示，则 G 中顶点 b 与 d 的度分别是



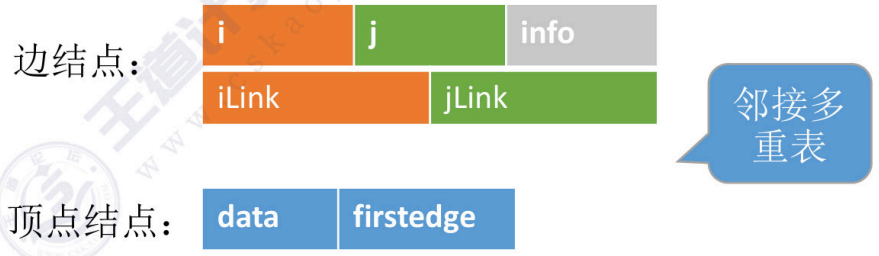

A. 0，2
B. 2，4
C. 2，5
D. 3，4

王道知识与解析 · 原解析已定位

【对应知识】6.2.3+6.2.4 十字链表、邻接多重表 · 原PDF第11页
原课件 6.2.3+6.2.4 十字链表、邻接多重表 · PDF p11

# 知识回顾与重要考点

| | 邻接矩阵 | 邻接表 | 十字链表 | 邻接多重表 |
|---|---|---|---|---|
| 空间复杂度 | $O(\|V\|^2)$ | 无向图 $O(\|V\| + 2\|E\|)$<br>有向图$O(\|V\| + \|E\|)$ | $O(\|V\| + \|E\|)$ | $O(\|V\| + \|E\|)$ |
| 找相邻边 | 遍历对应行或列<br>时间复杂度为$O(\|V\|)$ | 找有向图的入边必须遍历整个邻接表 | 很方便 | 很方便 |
| 删除边或顶点 | 删除边很方便，删除顶点需要大量移动数据 | 无向图中删除边或顶点都不方便 | 很方便 | 很方便 |
| 适用于 | 稠密图 | 稀疏图和其他 | 只能存有向图 | 只能存无向图 |
| 表示方式 | 唯一 | 不唯一 | 不唯一 | 不唯一 |




王道考研/CSKAOYAN.COM

对应原题本第99页；点击此行返回题目。

【原解析】王道2027数据结构 · 原PDF第223页
王道2027数据结构 §6.2 选择题20原解析

**20．B**

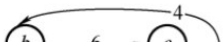
在邻接多重表中，统计一个顶点的入度、出度或度数（入度+出度）时，只需考虑所有的边结点。下图是邻接多重表的边结点的结构：

| ivex | ilink | jvex | jlink | info |
|---|---|---|---|---|

其中，ivex表示弧头的编号，ilink域指向下一个弧头相同的边结点，jvex表示弧尾的编号，jlink域指向下一个弧尾相同的边结点。根据定义可知：求$x$号顶点的入度时，只需统计jvex域等于$x$的边结点个数；求$x$号顶点的出度时，只需统计ivex域等于$x$的边结点个数；$x$号顶点的度

【原解析】王道2027数据结构 · 原PDF第224页
王道2027数据结构 §6.2 选择题20原解析

数为入度和出度之和。顶点$b$的编号为1，统计可知：入度为1，出度为1，度数为2。顶点$d$的编号为3，统计可知：入度为1，出度为3，度数为4。



# 2024-05



数据结构　单项选择题 · 2 分

下列数据结构中，不适合直接使用折半查找的是

I. 有序链表
II. 无序数组
III. 有序静态链表
IV. 无序静态链表

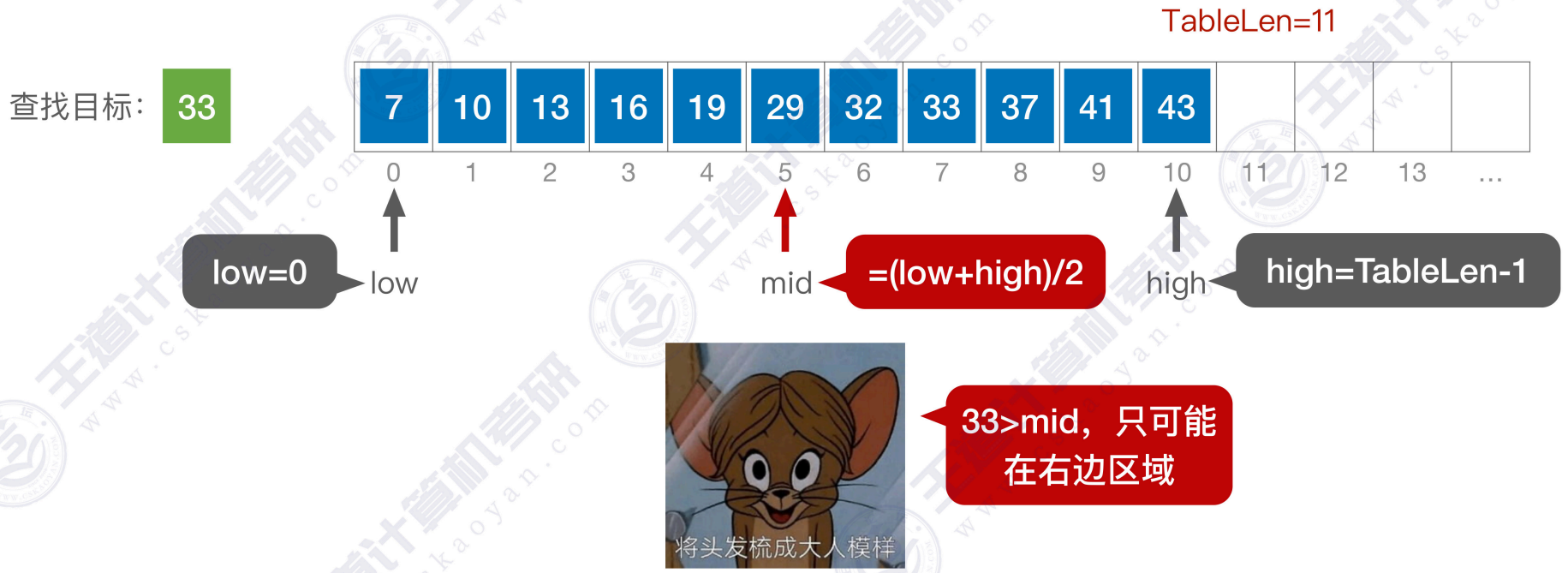
A. 仅I、III
B. 仅II、IV
C. 仅II、III、IV
D. I、II、III、IV

王道知识与解析 · 原解析已定位

【对应知识】7.2.2 折半查找 · 原PDF第3页
原课件 7.2.2 折半查找 · PDF p3

# 折半查找的算法思想

折半查找，又称“二分查找”，仅适用于有序的顺序表。




王道考研/CSKAOYAN.COM

【原解析】王道2027数据结构 · 原PDF第287页
王道2027数据结构 §7.2 选择题25原解析

**25．D**

折半查找必须满足两个条件：① 数组（或顺序表），折半查找的上一次查找和本次查找可能相

【原解析】王道2027数据结构 · 原PDF第288页
王道2027数据结构 §7.2 选择题25原解析

隔很远的距离，如依次查找下标为 $n/2, n/4, n/8, \cdots$ 的元素，若采用链表（或静态链表），则会使得时间复杂度非常高。② 有序，只有在有序的情况下才能根据上一次的比较情况舍弃一半的序列。

对应原题本第100页；点击此行返回题目。

# 2024-06



数据结构 · 单项选择题 · 2 分

KMP算法使用修正后的next数组进行模式匹配，模式串S="aabaab"，当主串中某字符与S中某字符失配时，S将向右滑动的最长距离是

A. 5
B. 4
C. 3
D. 2

## 王道知识与解析 · 原解析已定位

【对应知识】王道2027数据结构 · 原PDF第126页
王道2027数据结构 · KMP的next数组手算 · PDF p126

还有一种特殊情况：若模式串的第一个字符即与主串当前字符失配，则已匹配字符数为 0，按规则将模式串整体右移一位，并从主串下一个位置开始新一轮匹配。

**2. next 数组的手算方法**

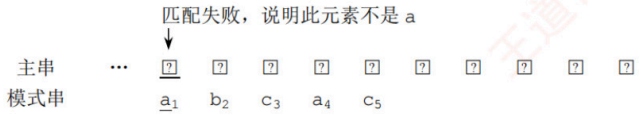
在实际匹配过程中，模式串在内存中的存储位置是固定的，并不会真正“滑动”；发生变化的只是指针。前文通过模拟滑动的方式，是为了直观地帮助读者理解 KMP 算法的匹配逻辑。

考点追踪 ▶ KMP 算法中指针变化/比较次数的分析（2015、2019）

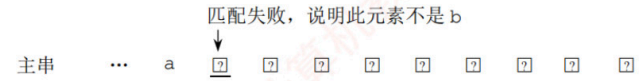
KMP算法的核心特性是：**每趟匹配失败时，主串指针i不回溯，仅模式串指针j调整**。为此，可定义一个next数组，**next[j]的含义**是：当模式串的第j个字符失配时，下一轮应从模式串的第next[j]个位置继续与主串当前字符比较。

下面介绍一种手算 next 数组的方法，仍以模式串'abcac'为例。

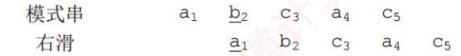
**第 1 个字符失配**时：令 next[1]=0，然后指针 i 加 1，指针 j 重置为 1，即下一轮将模式串的第 1 个位置与主串当前位置的下一位置进行比较（注意，图中下标为字符编号）。




**第 2 个字符失配**时：令 next[2]=1，模式串下次的比较位置为 1，相当于向右滑动 1 位（注，next[1]=0、next[2]=1 是固定规则）。

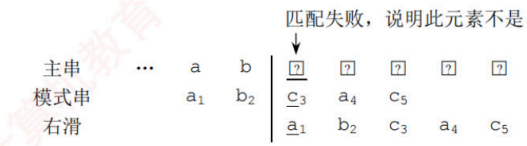



【对应知识】王道2027数据结构 · 原PDF第126页（续）
王道2027数据结构 · KMP的next数组手算 · PDF p126

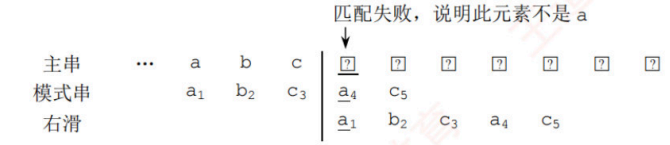



在后续的手算过程中，可在失配位置前画一条分界线，尝试将模式串向右移动，直到分界线左侧的部分能够首尾对齐（存在相等的前后缀），或模式串完全跨过分界线为止。

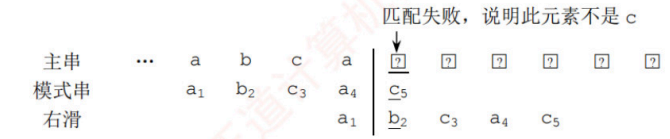
**第 3 个字符失配**时：模式串下次的比较位置为 1，即 next[3]=1，相当于向右滑动 2 位。




**第 4 个字符失配**时：模式串下次的比较位置为 1，即 next[4]=1，相当于向右滑动 3 位。




**第 5 个字符失配**时：模式串下次的比较位置为 2，即 next[5]=2，相当于向右滑动 3 位。



【对应知识】王道2027数据结构 · 原PDF第129页
王道2027数据结构 · KMP优化与nextval · PDF p129

```
int Index_KMP(SString S,SString T,int next[]){
    int i=1, j=1;
    while(i<=S.length&&j<=T.length){
        if(j==0||S.ch[i]==T.ch[j]){
            ++i; ++j;              //继续比较后继字符
        }
        else
            j=next[j];             //模式串向右滑动
    }
    if(j>T.length)
        return i-T.length;         //匹配成功
    else
        return 0;
}
```

尽管简单模式匹配的最坏时间复杂度为 $O(mn)$，而 KMP 算法可达到 $O(m+n)$，但在一般情况下，简单模式匹配的平均执行时间往往接近 $O(m+n)$，因此至今仍被采用。KMP 算法仅在主串与模式串存在大量“部分匹配”时才显著优于暴力匹配，其核心优点在于主串指针不回溯。

### 4.2.3 KMP 算法的进一步优化

考点追踪 ▶▶ nextval 数组的计算（2024）

前面定义的 next 数组在某些情况下仍存在缺陷，还可进一步优化。图 4.5 展示了一个典型示例：模式串'aaaab'与主串'aaabaaaab'进行匹配。

| 主串 | a | a | a | b | a | a | a | a | b |
|---|---|---|---|---|---|---|---|---|---|
| 模式串 | a | a | a | a | b | | | | |
| j | 1 | 2 | 3 | 4 | 5 | | | | |
| next[j] | 0 | 1 | 2 | 3 | 4 | | | | |
| nextval[j] | 0 | 0 | 0 | 0 | 4 | | | | |

图 4.5 KMP 算法的进一步优化示例

当 i=4、j=4 时，主串字符 $s_4$ 与模式串字符 $p_4$（b≠a）失配。若使用之前的 next 数组，则还需依次进行 $s_4$ 与 $p_3$、$s_4$ 与 $p_2$、$s_4$ 与 $p_1$ 的三次比较。然而，由于 next[4]=3 且 $p_3=p_4=a$，同理 next[3]=2 且 $p_2=p_3=a$，next[2]=1 且 $p_1=p_2=a$，由于这些回退位置上的字符均与 $p_4$ 相同，继续比较只会重复同样的失配过程，造成不必要的开销。

问题根源在于：**不应出现** $p_j=p_{next[j]}$。理由是：当 $p_j\neq s_i$ 时，下一轮将用 $p_{next[j]}$ 与 $s_i$ 比较；若 $p_{next[j]}=p_j$，则相当于用与 $p_j$ 相同的字符去匹配已知不等的 $s_i$，结果必定仍是失配。

*那么，若出现 $p_j=p_{next[j]}$，应如何处理？*

此时应不断将 next[j]替换为 next[next[j]]，直到找到某个位置 k，使得 $p_j\neq p_k$ 或 k=0 为止，修正后的新数组称为 **nextval 数组**。计算 nextval 数组的算法如下（匹配算法不变）。

```
void get_nextval(SString T,int nextval[]){
    int i=1, j=0;
    nextval[1]=0;
    while(i<T.length){
        if(j==0||T.ch[i]==T.ch[j]){
            ++i; ++j;
            if(T.ch[i]!=T.ch[j])  nextval[i]=j;
            else  nextval[i]=nextval[j];
        }
        else
            j=nextval[j];
    }
```

对应原题本第101页；点击此行返回题目。

【对应知识】王道2027数据结构 · 原PDF第129页（续）
王道2027数据结构 · KMP优化与nextval · PDF p129

```
        }
```

【原解析】王道2027数据结构 · 原PDF第133页
王道2027数据结构 §4.2 选择题12原解析

**12．A**

假设位序从 0 开始，计算出 nextval 数组。当比较到 S[j] 失配时，模式串向右滑动的距离为 j－nextval[j]（0≤j≤5），由下图可知，当 j=4 时向右滑动的距离最长，此时距离为 5。

| 编号 j | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| S[j] | a | a | b | a | a | b |
| next[j] | -1 | 0 | 1 | 0 | 1 | 2 |
| nextval[j] | -1 | -1 | 1 | -1 | -1 | 1 |
| j-nextval[j] | 1 | 2 | 1 | 4 | 5 | 4 |

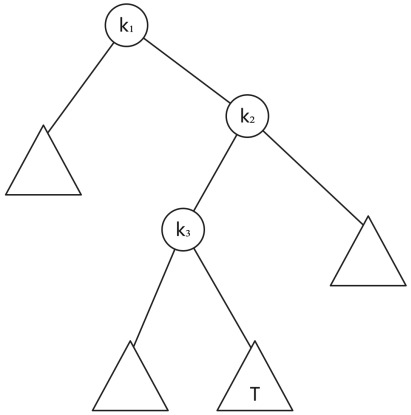
对应原题本第101页；点击此行返回题目。

# 2024-07



数据结构 · 单项选择题 · 2 分

一棵二叉搜索树如题7图所示，k1、k2、k3分别是对应结点中保存的关键字。子树T的任一结点中保存的关键字x满足的是




A. x < k1
B. x > k2
C. k1 < x < k3
D. k3 < x < k2

王道知识与解析 · 原解析已定位

【对应知识】7.3.1 二叉排序树 · 原PDF第3页
原课件 7.3.1 二叉排序树 · PDF p3

# 二叉排序树的定义

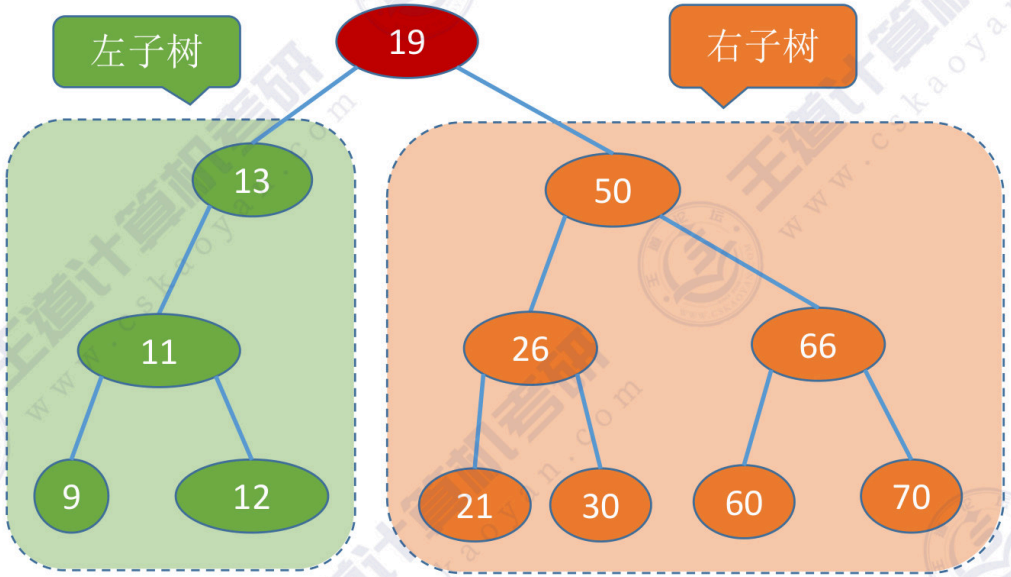
二叉排序树可用于元素的有序组织、搜索

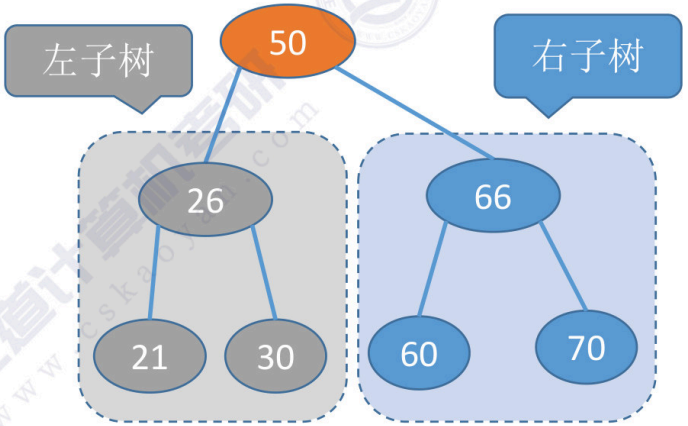
二叉排序树，又称二叉查找树（BST，Binary Search Tree）
一棵二叉树或者是空二叉树，或者是具有如下性质的二叉树：
左子树上所有结点的关键字均小于根结点的关键字；
右子树上所有结点的关键字均大于根结点的关键字。
左子树和右子树又各是一棵二叉排序树。

→ 左子树结点值 < 根结点值 < 右子树结点值

↓

进行中序遍历，可以得到一个递增的有序序列




王道考研/CSKAOYAN.COM

对应原题本第102页；点击此行返回题目。

【原解析】王道2027数据结构 · 原PDF第313页
王道2027数据结构 §7.3 选择题34原解析

**34.** D

在二叉搜索树中，每个结点的左子树中的所有关键字都小于该结点的关键字，右子树中的所有关键字都大于该结点的关键字。$k_2$、$k_3$所在结点都在$k_1$所在结点的右子树中，因此$k_2 > k_1$、$k_3 > k_1$；$k_3$所在结点在$k_2$所在结点的左子树中，因此$k_3 < k_2$；子树$T$是$k_3$所在结点的右子树，因此子树$T$中任意结点的关键字$x$满足$x > k_3$。综合可得，$k_3 < x < k_2$。

对应原题本第102页；点击此行返回题目。

# 2024-08



数据结构 · 单项选择题 · 2 分

使用快速排序算法对含n(n≥3)个元素的数组M进行排序，若第一趟排序将M中除枢轴外的n－1个元素划分为均不为空的P和Q两块，则下列叙述中，正确的是

A. P与Q块间有序
B. P与Q均块内有序
C. P和Q的元素个数大致相等
D. P中和Q中均不存在相等的元素

王道知识与解析 · 原解析已定位

【对应知识】8.3.2 快速排序 · 原PDF第132页
原课件 8.3.2 快速排序 · PDF p132

# 知识回顾与重要考点

算法思想：在待排序表L[1...n]中任取一个元素pivot作为枢轴（或基准，通常取首元素），通过一趟排序将待排序表划分为独立的两部分L[1...k−1]和L[k+1...n]，使得L[1...k−1]中的所有元素小于pivot，L[k+1...n]中的所有元素大于等于pivot，则pivot放在了其最终位置L(k)上，这个过程称为一次“划分”。然后分别递归地对两个子表重复上述过程，直至每部分内只有一个元素或空为止，即所有元素放在了其最终位置上。

注：408原题中说，对所有尚未确定最终位置的所有元素进行一遍处理称为“一趟”排序，因此一次“划分”≠一趟排序。一次划分可以确定一个元素的最终位置，而一趟排序也许可以确定多个元素的最终位置。

王道考研/CSKAOYAN.COM

对应原题本第103页；点击此行返回题目。

【原解析】王道2027数据结构 · 原PDF第360页
王道2027数据结构 §8.3 选择题20原解析

**20**. A

依题意，对数组 $M$ 进行第一趟快速排序后，会将 $M$ 划分为三部分：小于枢轴值的块 $P$，枢轴元素，大于枢轴值的块 $Q$。$P$ 和 $Q$ 块内都是无序的，但是 $P$ 块内的数据≤枢轴值，$Q$ 块内的数据≥枢轴值，因此 $P$ 块内的数据≤$Q$ 块内的数据，即 $P$ 与 $Q$ 块间有序。

对应原题本第103页；点击此行返回题目。

# 2024-09



数据结构　单项选择题 · 2 分

已知关键字序列28，22，20，19，8，12，15，5是大根堆（最大堆），对该堆进行两次删除操作后，得到的新堆是

A. 20，19，15，12，8，5
B. 20，19，15，5，8，12
C. 20，19，12，15，8，5
D. 20，19，8，12，15，5

## 王道知识与解析 · 原解析已定位

【对应知识】王道2027数据结构 · 原PDF第366页
王道2027数据结构 · 建堆及堆排序复杂度 · PDF p366

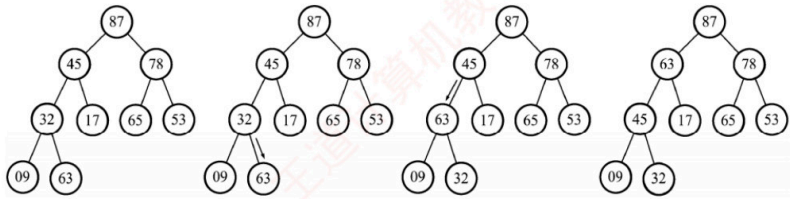



图 8.8　大根堆的插入操作示例

考点追踪 ▶▶ 堆在海量数据中选出最小 $k$ 个数的应用及效率分析（2022）

堆排序适用于大规模数据的排序场景，还常用于解决 Top-K 问题。例如，在 1 亿个数中选出最大的 100 个数。首先使用一个大小为 100 的数组，读入前 100 个数，构建小顶堆，而后依次读入后续数字，若小于堆顶则舍弃，否则用该数替换堆顶并向下调整，待数据读取完毕，堆中 100 个数为所求。此方法的时间复杂度为 $O(n\log_2 k)$，空间复杂度为 $O(k)$，效率远高于全排序。

堆排序算法的性能分析如下。

**空间效率**：仅使用了常数个辅助单元，空间复杂度为 $O(1)$。

**时间效率**：建堆的时间复杂度为 $O(n)$，随后需进行 $n-1$ 次堆顶输出与调整操作，每次调整的时间复杂度为 $O(\log_2 n)$。因此，堆排序在最好、最坏和平均情况下的时间复杂度均为 $O(n\log_2 n)$。

**稳定性**：在堆调整过程中，相等关键字的元素可能因交换而改变其原始相对位置，因此堆排序是不稳定的排序算法。例如，表 $L=\{1,\mathbf{2},2\}$，建堆时可能将 **2** 交换到堆顶，此时 $L=\{\mathbf{2},1,2\}$，最终排序序列为 $L=\{1,2,\mathbf{2}\}$，显然，2 与 **2** 的相对次序已发生变化。

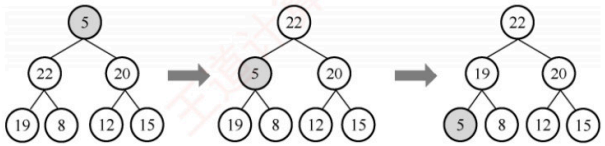
【对应知识】王道2027数据结构 · 原PDF第366页（续）
王道2027数据结构 · 建堆及堆排序复杂度 · PDF p366

**适用性**：堆排序依赖完全二叉树的随机访问特性，因此仅适用于顺序存储的线性表。

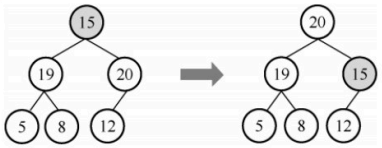
【原解析】王道2027数据结构 · 原PDF第372页
王道2027数据结构 §8.4 选择题21原解析

**21**. B

该序列已调整成大根堆，接下来进行第一次删除操作：删除 28 后，将 5 放入堆顶；然后自上而下调整。22 和 20 比较，较大者 22 与 5 比较，交换；19 和 8 比较，较大者 19 与 5 比较，交换。



接下来进行第二次删除操作：删除 22 后，将 15 放入堆顶；然后自上而下调整。19 和 20 比较，较大者 20 与 15 比较，交换；15 直接与其仅有的左孩子 12 比较，不交换。



因此，进行两次删除操作后，得到的新堆是 20, 19, 15, 5, 8, 12。

# 2024-10



数据结构 | 单项选择题 · 2 分

现有由关键字组成的3个有序序列（3，5）、（7，9）和（6），若按从左至右的次序选择有序序列进行二路归并排序，则关键字之间的总比较次数是

A. 3
B. 4
C. 5
D. 6

## 王道知识与解析 · 原解析已定位

【对应知识】王道2027数据结构 · 原PDF第376页
王道2027数据结构 · 二路归并排序与归并操作 · PDF p376

```
ElemType *B=(ElemType *)malloc((n+1)*sizeof(ElemType)); //辅助数组 B
void Merge(ElemType A[],int low,int mid,int high){
//表 A 的两段 A[low…mid]和 A[mid+1…high]各自有序，将它们合并成一个有序表
    int i,j,k;
    for(k=low;k<=high;k++)
        B[k]=A[k];                    //将 A 中所有元素复制到 B 中
    for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
        if(B[i]<=B[j])                //比较 B 的两个段中的元素
            A[k]=B[i++];              //将较小值复制到 A 中
        else
            A[k]=B[j++];
    }
    while(i<=mid)   A[k++]=B[i++];  //若第一个表未检测完，复制
    while(j<=high)  A[k++]=B[j++];  //若第二个表未检测完，复制
}
```

**注 意**

在上述代码中，最后两个 while 循环仅有一个会执行。

在非递归的归并排序中，一趟归并操作是指：将当前序列划分为若干长度为 $h$ 的有序段，然后两两归并，形成长度为 $2h$ 的有序段。若序列总长度为 $n$，则每趟需调用$\lceil n/(2h)\rceil$次 Merge()函数。整个排序过程共需进行$\lceil \log_2 n\rceil$趟归并。

递归形式的 2 路归并排序基于**分治策略**，其过程如下。

**分解**：将含 $n$ 个元素的待排序表从中间分为两个子表，每个子表含约 $n/2$ 个元素。

**递归求解**：对两个子表分别递归地进行归并排序。

**合并**：合并两个已排序的子表得到排序结果。

【对应知识】王道2027数据结构 · 原PDF第376页（续）
王道2027数据结构 · 二路归并排序与归并操作 · PDF p376

2 路归并排序的递归实现如下：

```
void MergeSort(ElemType A[],int low,int high){
    if(low<high){
        int mid=(low+high)/2;       //从中间划分两个子序列
        MergeSort(A,low,mid);       //对左侧子序列进行递归排序
        MergeSort(A,mid+1,high);    //对右侧子序列进行递归排序
        Merge(A,low,mid,high);      //归并
    }
}
```

考点追踪 ▶ 2 路归并比较次数的分析（2024）

2 路归并排序算法的性能分析如下：

**空间效率**：归并操作需要一个与待排序表等长的辅助数组，因此空间复杂度为 $O(n)$。

**时间效率**：每趟归并需要遍历所有 $n$ 个元素，时间复杂度为 $O(n)$；共需进行$\lceil \log_2 n\rceil$趟，总时间复杂度为 $O(n\log_2 n)$。

**稳定性**：在归并过程中，当两个元素关键字相等时，优先取自前一段，从而保持了相同关键字元素的相对次序。因此 2 路归并排序是稳定的排序算法。

**适用性**：归并排序仅需顺序访问元素，因此适用于顺序存储和链式存储的线性表。

**注 意**

一般而言，对 $N$ 个元素进行 $k$ 路归并排序时，排序的趟数 $m$ 满足 $k^m=N$，从而 $m=\log_k N$，又考虑到 $m$ 为整数，因此 $m=\lceil \log_k N\rceil$。这与前述的 2 路归并排序算法是一致的。

【原解析】王道2027数据结构 · 原PDF第384页
王道2027数据结构 §8.5 选择题20原解析

**20．C**

从左至右对这三个有序序列进行 2 路归并排序的过程如下。

① 合并(3, 5)和(7, 9)：比较 3 和 7（1 次），选择 3；比较 5 和 7（1 次），选择 5；剩余元素 7 和 9 直接加入。② 合并(3, 5, 7, 9)和(6)：比较 3 和 6（1 次），选择 3；比较 5 和 6（1 次），选择 5；比较 7 和 6（1 次），选择 6；剩余元素 7 和 9 直接加入。总共比较 5 次。

对应原题本第105页；点击此行返回题目。

# 2024-11



数据结构　单项选择题 · 2 分

在外排序中，利用败者树对初始为升序的归并段进行多路归并，败者树中记录“冠军”的结点保存的是

A. 最大关键字
B. 最小关键字
C. 最大关键字所在的归并段号
D. 最小关键字所在的归并段号

王道知识与解析 · 原解析已定位

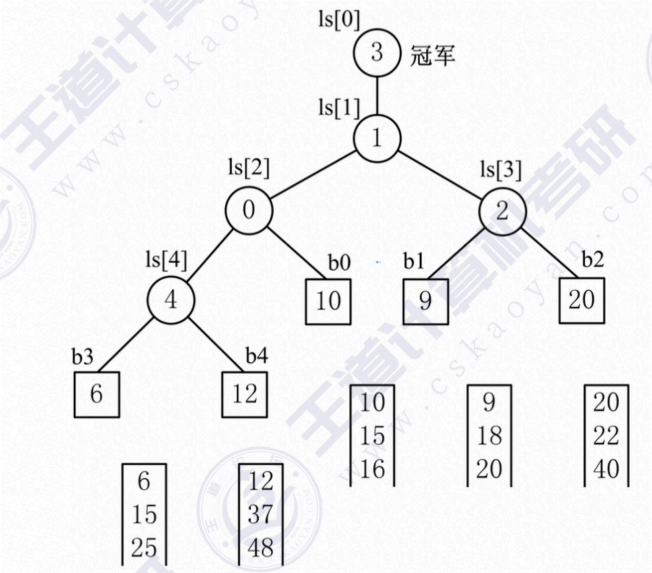
【对应知识】8.7.3 败者树 · 原PDF第35页
原课件 8.7.3 败者树 · PDF p35

# 知识回顾与重要考点

败者树解决的问题：使用多路平衡归并可减少归并趟数，但是用老土方法从 k 个归并段选出一个最小/最大元素需要对比关键字 k–1 次，构造败者树可以使关键字对比次数减少到 $\lceil log_2 k \rceil$

败者树可视为一棵完全二叉树（多了一个头头）。$k$个叶结点分别对应 k 个归并段中当前参加比较的元素，非叶子结点用来记忆左右子树中的“失败者”，而让胜者往上继续进行比较，一直到根结点。

**如何构造和使用败者树？——看图记忆**




王道考研/CSKAOYAN.COM

对应原题本第106页；点击此行返回题目。

【原解析】王道2027数据结构 · 原PDF第400页
王道2027数据结构 §8.7 选择题16原解析

**16．D**

利用败者树归并升序归并段，每次需要得到当前的最小关键字，因此记录“冠军”的结点保存的只能是最小关键字或最小关键字所在的归并段号。通过败者树找出最小关键字后，还需要找到该关键字所在的归并段，并移动段内元素，以继续比较下一个元素，下面分别进行讨论：① 假设记录“冠军”的结点保存的是最小关键字所在的归并段号，则能直接得到最小关键字及其所在的段，时间复杂度低。② 假设记录“冠军”的结点保存的是最小关键字，则查找其所在的段需要检索所有段的首元素是否与该最小关键字相等，时间复杂度高。为了提高效率，记录“冠军”的结点保存的是最小关键字所在的归并段号，而不是最小关键字。

对应原题本第106页；点击此行返回题目。

# 2024–12



组成原理 · 单项选择题 · 2 分

C 语言代码段如下。

```
int i=32777;
short si=i;
int j=si;
```

执行上述代码段后，j 的值是

A. -32 777
B. -32 759
C. 32 759
D. 32 777

## 王道知识与解析 · 原解析已定位

【原解析 + 知识】王道2027计算机组成原理 · 原PDF第63页
王道教材 §2.2 选择题33 原解析

**33**. B

$2^{15}=32768$，i = 32768 + 9 = 8000H + 9H = 8009H，32 位有符号数 i 的机器数为 0000 8009H。将 32 位有符号数 i 强制转换为 16 位有符号数 si，直接保留机器数的末 16 位即可，因此 si 的机器数为 8009H，真值为$-2^{15}+9=-32759$。将 16 位有符号数 si 强制转换为 32 位有符号数 j，采用符号扩展（si

【原解析 + 知识】王道2027计算机组成原理 · 原PDF第64页
王道教材 §2.2 选择题33 原解析

的符号为 1，因此高位补 1），j 的机器数变为 FFFF 8009H，对应的真值为−32759。

| | int i | short si | int j |
|---|---|---|---|
| 机器数 | 0000 8009H | 8009H | FFFF 8009H |
| 真值 | 32777 | −32759 | −32759 |

# 2024–13



组成原理 · 单项选择题 · 2 分

通常情况下，将汇编语言程序中实现特定功能的指令序列定义成一条伪指令 (pseudoinstruction)。下列选项中，CPU 能理解并直接执行的是

I. 伪指令

II. 微指令

III. 机器指令

IV. 汇编指令

A. 仅 I、IV

B. 仅 II、III

C. 仅 III、IV

D. 仅 I、III、IV

王道知识与解析 · 原解析已定位

此题在王道原资料中标注为2021年；已按完整题干、设问及四个选项核对，与本题一致。

【原解析 + 知识】王道2027计算机组成原理 · 原PDF第252页

王道教材 §5.4 选择题27 原解析

27. B

高级语言程序、汇编语言程序都需要通过翻译程序来处理，生成机器语言程序后才能被 CPU 执行。机器指令能被 CPU 理解并直接执行。微指令是 CPU 控制单元用于实现机器指令的更低层次的指令。在微程序控制的 CPU 中，一条机器指令对应一个微程序，微程序是微指令的有序序列，

【原解析 + 知识】王道2027计算机组成原理 · 原PDF第253页

王道教材 §5.4 选择题27 原解析

用来控制 CPU 实现机器指令的过程，因此微指令也能被 CPU 理解并直接执行。

# 2024-14



组成原理 · 单项选择题 · 2 分

某科学实验中，需要使用大量的整型参数，为了在保证表数精度的基础上提高运算速度，需要选择合理的数据表示方法。若整型参数 α、β 的取值范围分别为 $-2^{20}sim2^{20}$、$-2^{40}sim2^{40}$，则下列选项中，α、β 最适宜采用的数据表示方法分别是

A. 32位整数、32位整数
B. 单精度浮点数、单精度浮点数
C. 32位整数、双精度浮点数
D. 单精度浮点数、双精度浮点数

王道知识与解析 · 原解析已定位

【原解析 + 知识】王道2027计算机组成原理 · 原PDF第84页
王道教材 §2.3 选择题43 原解析

**43. C**

表示整数时，相同位数的浮点型的精度不如整型，因此在能满足 $\alpha$、$\beta$ 的取值范围的前提下，应优先选择整型；否则，才选择浮点型。32 位补码整数的表示范围为 $-2^{31}\sim2^{31}-1$，满足 $\alpha$ 的取值范围，因此 $\alpha$ 应采用 32 位整数。双精度浮点数的精度比单精度浮点数的更高，$-2^{40}\sim2^{40}$ 超出了单精度浮点数可以表示的精度（由尾数位数+隐含位决定），因此 $\beta$ 应采用双精度浮点数。

# 2024–15



组成原理 · 单项选择题 · 2 分

下列关于整数乘法运算的叙述中，错误的是

A. 用阵列乘法器实现的乘运算可以在一个时钟周期内完成
B. 用 ALU 和移位器实现的乘运算无法在一个时钟周期内完成
C. 变量与常数的乘运算可编译优化为若干条移位及加/减运算指令
D. 两个变量的乘运算无法编译转换为移位及加法等指令的循环实现

王道知识与解析 · 原解析已定位

【原解析 + 知识】王道2027计算机组成原理 · 原PDF第64页
王道教材 §2.2 选择题34 原解析

**34．D**

阵列乘法器中的所有部分积同时产生并组成一个阵列，运用多操作数相加就能得到最终的积，因此可在一个时钟周期内完成。用 ALU 和移位器实现的乘运算通常采用串行的乘法算法，需要多个时钟周期才能完成。当一个乘数是常数时，编译器可将乘运算优化为若干移位和加减运算指令。两个变量的乘运算可通过移位和加法等指令循环实现，选项 D 错误。

# 2024–16



组成原理 · 单项选择题 · 2 分

对于页式虚拟存储管理系统，下列关于存储器层次结构的叙述中，错误的是

A. Cache-主存层次的交换单位为主存块，主存-外存层次的交换单位为页
B. Cache-主存层次替换算法由硬件实现，主存-外存层次替换算法由软件实现
C. Cache-主存层次可采用回写法写策略，主存-外存层次通常采用回写法写策略
D. Cache-主存层次可采用直接映射方式，主存-外存层次通常采用直接映射方式

## 王道知识与解析 · 原解析已定位

【原解析 + 知识】王道2027计算机组成原理 · 原PDF第154页
王道教材 §3.6 选择题18 原解析

**18.** D

Cache 与主存之间交换的是主存块，主存与外存之间交换的是页。Cache–主存层次和主存–外存层次的区别在于前者主要解决速度不匹配问题，用软件实现会影响速度，因此 Cache–主存层次替

【原解析 + 知识】王道2027计算机组成原理 · 原PDF第155页
王道教材 §3.6 选择题18 原解析

换算法由硬件实现；而主存–外存层次替换算法由软件实现。Cache–主存层次可采用回写法或全写法；主存–外存层次通常采用回写法，即页面被修改后，仅当被换出时才写回外存，访问外存的代价很大，采用全写法的开销过高。访问外存的代价很大，提高命中率是关键，因此主存–外存层次通常采用全相联映射；而 Cache–主存层次可采用直接映射、组相联或全相联，选项 D 错误。

# 2024–17



组成原理 · 单项选择题 · 2 分

某计算机按字节编址，采用页式虚拟存储管理方式，虚拟地址为32位，主存地址为30位，页大小为1 KB。若TLB共有32个表项，采用4路组相联映射方式，则TLB表项中标记字段的位数至少是

A. 17
B. 18
C. 19
D. 20

王道知识与解析 · 原解析已定位

【原解析 + 知识】王道2027计算机组成原理 · 原PDF第155页
王道教材 §3.6 选择题19 原解析

**19．C**

按字节编址，页大小为 $2^{10}$B，因此页内地址占 10 位；TLB 有 32 个表项，采用 4 路组相联映射，被分为 $2^3=8$ 组， 因此 TLB 组号占 3 位；于是，标记字段的位数至少是 $32-3-10=19$。

# 2024–18



组成原理 · 单项选择题 · 2 分

下列事件中，不是在MMU地址转换过程中检测的是

A. 访问越权
B. Cache缺失
C. 页面缺失
D. TLB缺失

王道知识与解析 · 原解析已定位

【原解析 + 知识】王道2027计算机组成原理 · 原PDF第155页
王道教材 §3.6 选择题20 原解析

**20．B**

在地址转换的过程中，MMU 会检查页表项的访问权限，以确保进程有权访问某个页面，否则就会访问越权。为了获得对应的页表项，先查找 TLB，若找不到，则 TLB 缺失，然后查找页表，若找不到，则页面缺失。访问 Cache 是在获得物理地址后使用物理地址存取数据的过程中才执行的操作，而 MMU 地址转换过程是在获得物理地址之前进行的，选项 B 错误。

# 2024–19



组成原理 · 单项选择题 · 2 分

对于采用“取指、译码/取数、执行、访存、写回”5段流水线的RISC数据通路，下列关于指令流水线数据冒险处理的叙述中，错误的是

A. 相邻两条指令中的操作数相关可能引起数据冒险
B. 在数据相关的指令间插入“气泡”能避免数据冒险
C. 所有数据冒险都可以通过加入转发（旁路）电路解决
D. 所有数据冒险都能通过调整指令顺序和插入nop指令解决

王道知识与解析 · 原解析已定位

【原解析 + 知识】王道2027计算机组成原理 · 原PDF第276页
王道教材 §5.6 选择题31 原解析

**31. C**

对于 load-use 数据冒险，即后一条指令 y 需要使用前一条访存指令 x 从主存中取出的数据，由于指令 y 中所需的数据在指令 x 的访存阶段（MEM）结束时才可使用，即使有转发电路，指令 y 在执行阶段也无法获得所需的数据，只能通过插入 nop 指令或调整指令顺序解决。

# 2024-20



组成原理 · 单项选择题 · 2 分

某存储器总线的时钟频率为420 MHz，总线宽度为64位，每个时钟周期传送2次数据；其总线事务支持突发传送方式，最多传送8次数据，第1个时钟周期传送地址和读/写命令，从第4个至第7个时钟周期连续传送8次数据。该总线的总线带宽（最大数据传输率）为

A. 3.84 GB/s
B. 6.72 GB/s
C. 30.72 GB/s
D. 53.76 GB/s

## 王道知识与解析 · 原解析已定位

【原解析 + 知识】王道2027计算机组成原理 · 原PDF第294页
王道教材 §6.1 选择题24 原解析

**24. B**

总线带宽（最大数据传输率）是理想情况（所有总线周期都在传送数据）下，不需考虑每个总

【原解析 + 知识】王道2027计算机组成原理 · 原PDF第295页
王道教材 §6.1 选择题24 原解析

线事务的具体情况，而计算（平均）数据传输率才需考虑每个总线事务的具体情况。因此，题中“其总线事务支持……连续传送 8 次数据。”这句话属于干扰条件。根据定义，总线带宽 = 总线宽度×总线时钟频率×每个时钟周期传送数据的次数 = 64bit×420MHz×2 = 6.72GB/s。

# 2024–21



组成原理 · 单项选择题 · 2 分

下列关于中断I/O方式的叙述中，错误的是

A. 中断屏蔽字用于确定中断响应的优先级
B. 保存断点和程序状态字在中断响应阶段完成
C. 保存通用寄存器和设置新中断屏蔽字由软件实现
D. 单重中断方式下中断处理时CPU处于关中断状态

## 王道知识与解析 · 原解析已定位

【原解析 + 知识】王道2027计算机组成原理 · 原PDF第334页
王道教材 §7.3 选择题52 原解析

**52. A**

中断优先级包括响应优先级和处理优先级。响应优先级由硬件线路或查询程序的查询顺序确

【原解析 + 知识】王道2027计算机组成原理 · 原PDF第335页
王道教材 §7.3 选择题52 原解析

定，不可动态改变；处理优先级由中断屏蔽字确定，可灵活改变。

# 2024–22



组成原理 · 单项选择题 · 2 分

DMA控制I/O方式下，设备的输入/输出由DMA控制器控制完成，此时，DMA控制器控制的数据传输通路位于

A. CPU和主存之间
B. CPU和DMA控制器之间
C. 设备接口和主存之间
D. 设备接口和DMA控制器之间

王道知识与解析 · 原解析已定位

【原解析 + 知识】王道2027计算机组成原理 · 原PDF第335页
王道教材 §7.3 选择题53 原解析

**53. C**

DMA 的传送过程：① 预处理：CPU 完成一些必要的准备工作，由 DMA 控制器向 CPU 发总线请求。② 数据传送：DMA 控制器接管总线后，在设备接口和主存之间进行数据传送，此阶段由 DMA 控制器控制。③ 后处理：传送结束后，DMA 控制器向 CPU 发送中断信号，做结束处理。

# 2024-23



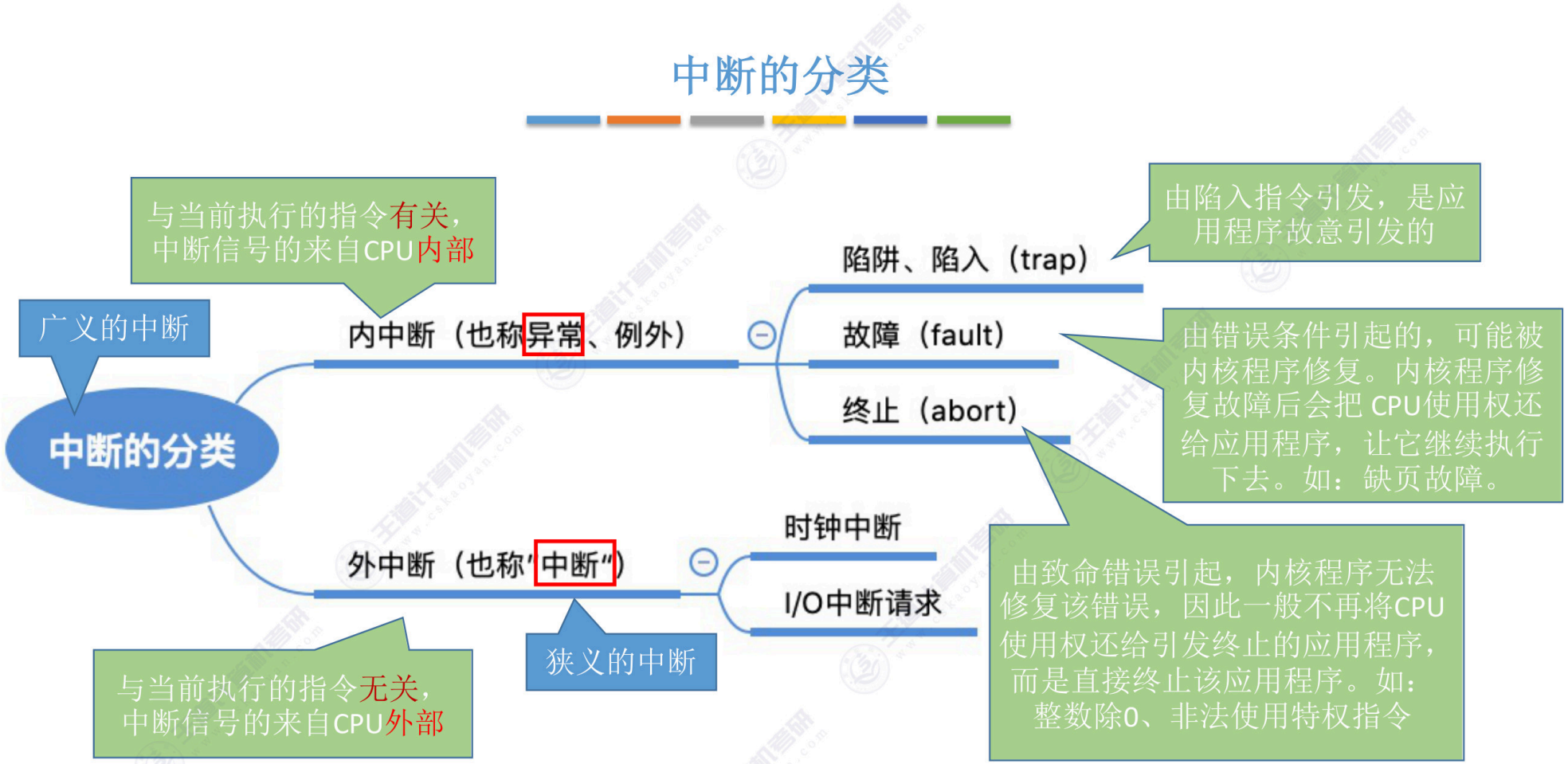
操作系统　单项选择题 · 2 分

下列关于中断、异常和系统调用的叙述中，错误的是

A. 中断或异常发生时，CPU处于内核态
B. 每个系统调用都有对应的内核服务例程
C. 中断处理程序开始执行时，CPU处于内核态
D. 系统添加新类型设备时，需注册相应的中断服务例程

王道知识与解析 · 原解析已定位

【对应知识】1.3.2 中断和异常 · 原PDF第8页
王道原课件：1.3.2 中断和异常




大多数的教材、试卷中，“中断”特指狭义的中断，即外中断。而内中断一般称为“异常”

王道考研/CSKAOYAN.COM

对应原题本第118页；点击此行返回题目。

【原解析 + 知识】王道2027操作系统 · 原PDF第38页
王道2027操作系统 §1.3 选择题33原解析

**33**. A

当中断或异常发生时，CPU既可能处于内核态，又可能处于用户态，具体取决于当时CPU正在处理的任务，选项A错误。不同的系统调用对应不同的内核服务例程，选项B正确。在中断响应阶段，若CPU处于用户态，则需要切换到内核态，因此在中断处理阶段，CPU一定处于内核态，选项C正确。设备种类繁多，计算机不可能事先准备好所有设备对应的中断服务例程（实际上属于设备驱动程序），因此当系统添加新类型的设备时，需要注册相应的中断服务例程。

对应原题本第118页；点击此行返回题目。

# 2024-24



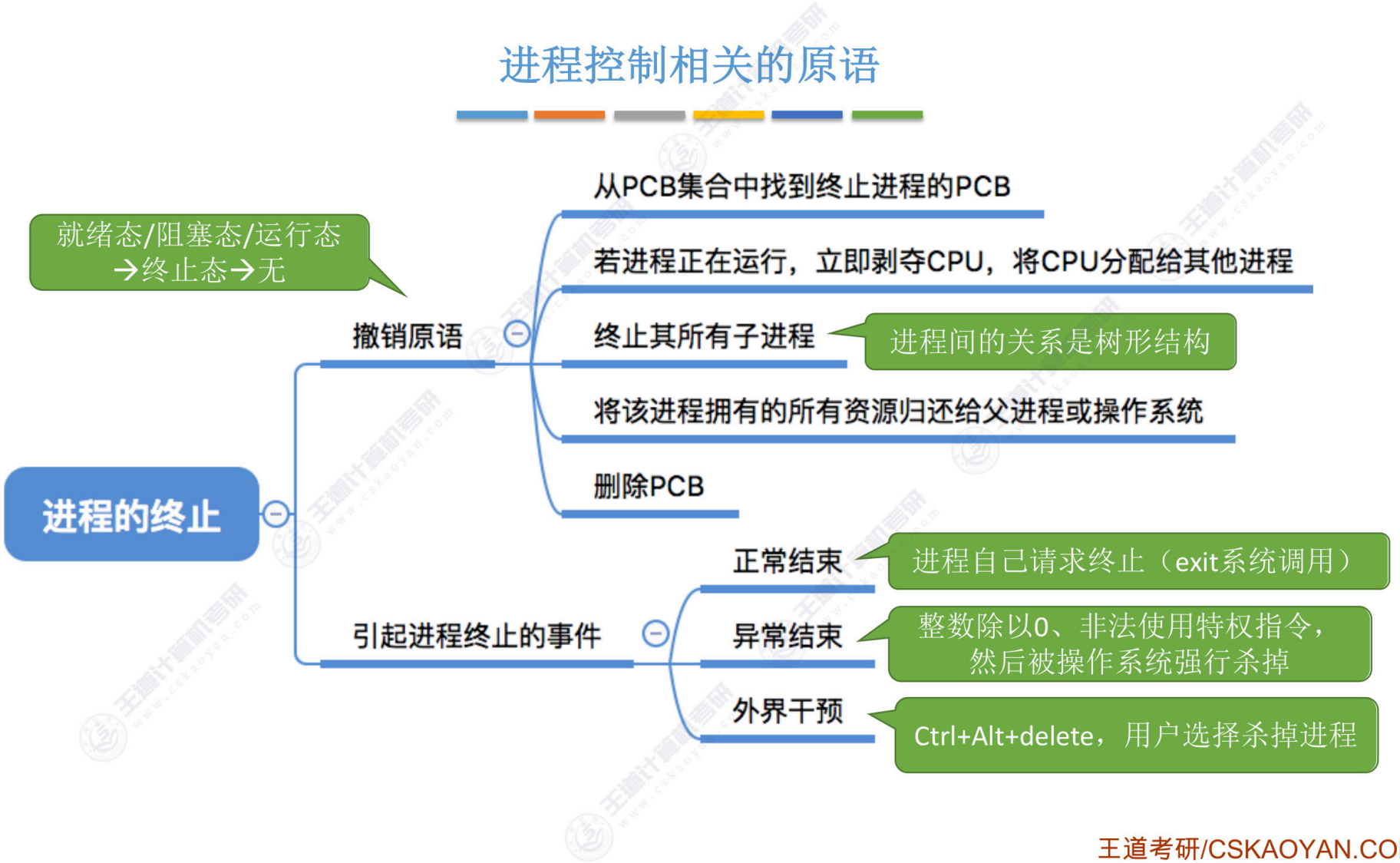
操作系统 · 单项选择题 · 2 分

下列选项中，操作系统在终止进程时不一定执行的是

A. 终止子进程
B. 回收进程占用的设备
C. 释放进程控制块
D. 回收为进程分配的内存

王道知识与解析 · 原解析已定位

【对应知识】2.1.4 进程控制 · 原PDF第9页
王道原课件：2.1.4 进程控制




王道考研/CSKAOYAN.COM

对应原题本第119页；点击此行返回题目。

【原解析 + 知识】王道2027操作系统 · 原PDF第77页
王道2027操作系统 §2.1 选择题72原解析

**72．A**

当操作系统终止进程时，所有的进程资源（如内存空间、进程控制块、设备、打开文件、I/O缓冲区等）都会被释放。有些系统不允许子进程在父进程已终止的情况下存在，对于这类系统，

【原解析 + 知识】王道2027操作系统 · 原PDF第78页
王道2027操作系统 §2.1 选择题72原解析

若一个进程终止，则它的所有子进程也终止，这种现象称为级联终止。但是，不是所有操作系统都是这么设计的，因此终止子进程不一定在终止进程时执行。

对应原题本第119页；点击此行返回题目。

# 2024–25



操作系统 · 单项选择题 · 2 分

在支持页式存储管理的系统中，进程切换时操作系统需要执行的操作是

Ⅰ. 更新程序计数器的值

Ⅱ. 更新栈基址寄存器的值

Ⅲ. 更新页表基地址寄存器的值

A. 仅Ⅲ

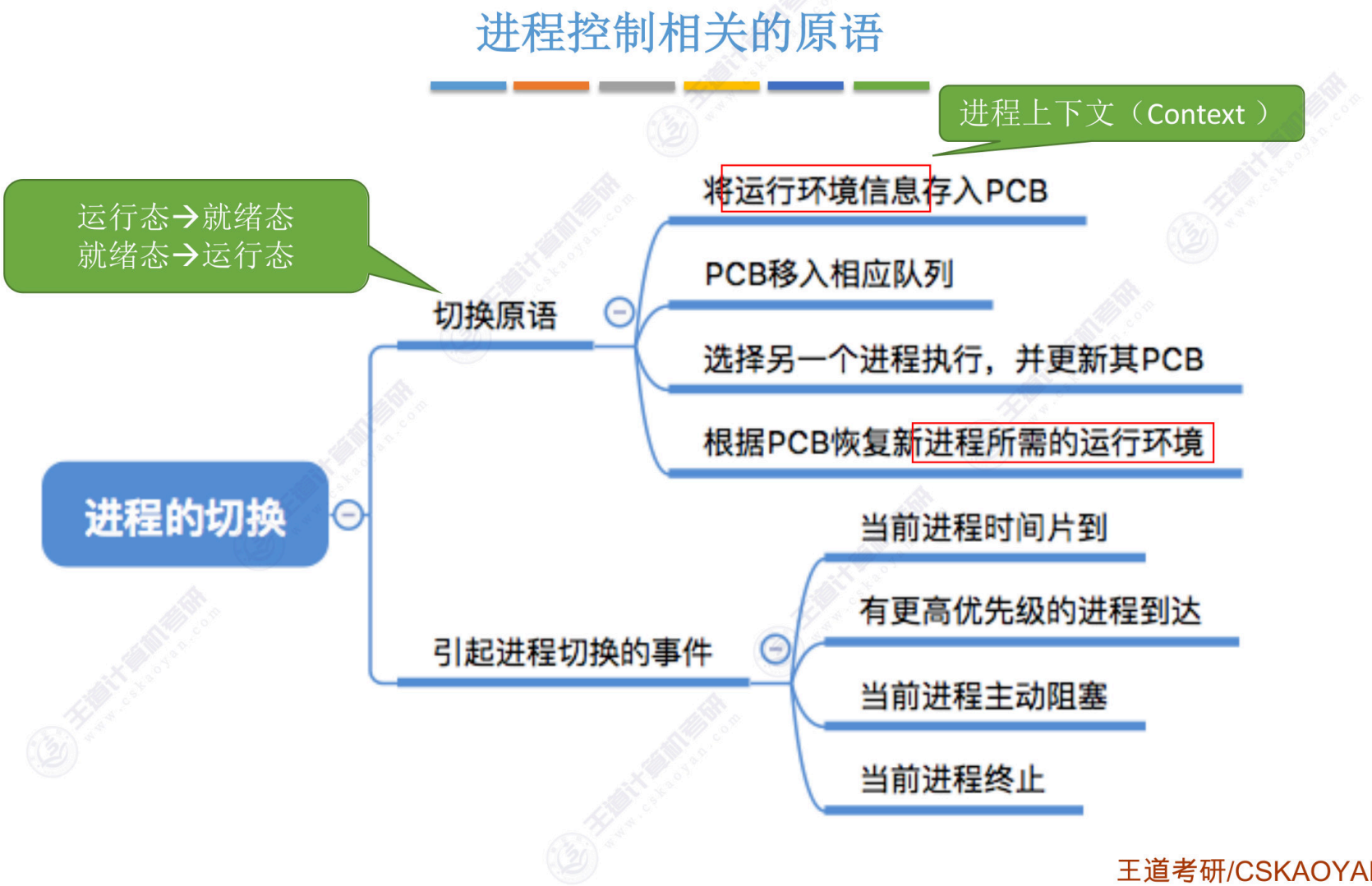
B. 仅Ⅰ、Ⅱ

C. 仅Ⅰ、Ⅲ

D. Ⅰ、Ⅱ、Ⅲ

王道知识与解析 · 原解析已定位

【对应知识】2.1.4 进程控制 · 原PDF第11页
王道原课件：2.1.4 进程控制




王道考研/CSKAOYAN.COM

对应原题本第120页；点击此行返回题目。

【原解析 + 知识】王道2027操作系统 · 原PDF第105页
王道2027操作系统 §2.2 选择题55原解析

**55.** D

进程切换时，需要进行上下文切换，主要完成：① 将当前 CPU 中各寄存器的内容保存到当前进程的 PCB 中；② 将新进程的现场信息装入 CPU 的各个寄存器。③ 将控制转至新进程执行，即更新程序计数器（PC）的值。栈基址寄存器用来指明系统内核栈位置的寄存器，页表基址寄存器用来指明当前进程顶级页表基地址的寄存器，属于进程控制信息。因此，说法 I、II、III 均需要更新。

对应原题本第120页；点击此行返回题目。

# 2024–26



操作系统　单项选择题 · 2 分

文件系统需占用部分外存空间记录空闲块位置。下列方法中，占用外存空间的大小与当前空闲块数量无关的是

A. 位图法
B. 空闲表法
C. 成组链接法
D. 空闲链表法

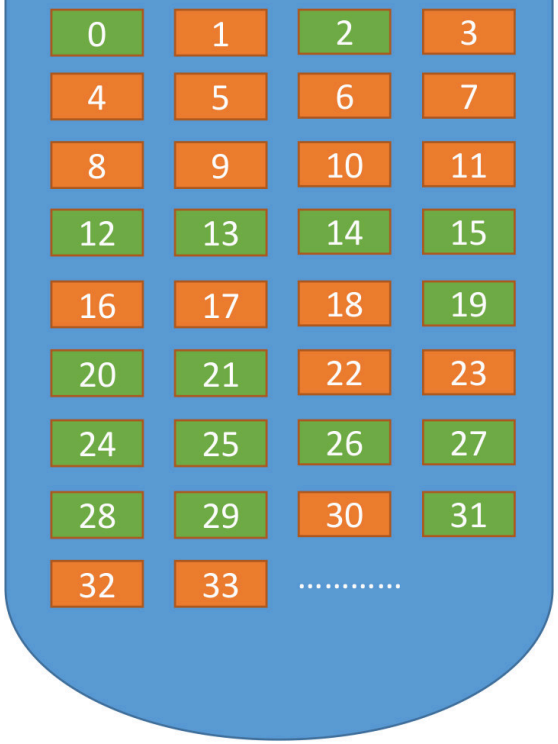
王道知识与解析 · 原解析已定位

【对应知识】4.1_6_文件存储空间管理 · 原PDF第11页
王道原课件：4.1_6_文件存储空间管理

# 存储空间管理——位示图法

(0,1) → b=16*0+1=1
(1,10) → b=16*1+10=26

b=13 → i=13/16=0, j=13%16=13
b=31 → i=31/16=1, j=31%16=15




| 字号 \ 位号 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| 2 | 1 | 1 | … | | | | | | | | | | | | | |
| … | | | | | | | | | | | | | | | | |

位示图：每个二进制位对应一个盘块。在本例中，“0”代表盘块空闲，“1”代表盘块已分配。位示图一般用连续的“字”来表示，如本例中一个字的字长是16位，字中的每一位对应一个盘块。因此可以用（字号，位号）对应一个盘块号。当然有的题目中也描述为（行号，列号）

***重要重要重要：要能自己推出盘块号与（字号,位号）相互转换的公式。***

注意题目条件：盘块号、字号、位号到底是从0开始还是从1开始
如本例中盘块号、字号、位号从0开始，若n表示字长，则...

(字号, 位号)=(i, j) 的二进制位对应的 盘块号 b = ni + j

b号盘块对应的字号 i = b/n，位号 j = b%n

注：王道课本上给出的是盘块号、字号、位号都从1开始的算法。

王道考研/CSKAOYAN.COM

对应原题本第121页；点击此行返回题目。

【原解析 + 知识】王道2027操作系统 · 原PDF第315页
王道2027操作系统 §4.3 选择题17原解析

**17. A**

位图法利用一个二进制位来表示磁盘中一个盘块的使用情况，磁盘上的所有盘块都有一个二进制位与之对应。当其值为“0”时，表示对应的盘块空闲；当其值为“1”时，表示已分配。位图所占空间的大小只取决于外存空间的总大小（盘块数量），而与当前空闲块数量无关。

对应原题本第121页；点击此行返回题目。

# 2024–27



操作系统　单项选择题 · 2 分

下列算法中，每次回收分区时仅合并大小相等的空闲分区的是

A. 伙伴算法
B. 最佳适应算法
C. 最坏适应算法
D. 首次适应算法

王道知识与解析 · 原解析已定位

【原解析 + 知识】王道2027操作系统 · 原PDF第220页
王道2027操作系统 §3.1 选择题72原解析

**72. A**

伙伴算法在回收一个大小为 $2^i$ 的空闲分区时，会检查其**伙伴分区**（地址相邻、大小同为 $2^i$ 的分区）是否也空闲。若是，则将两者**合并**为一个大小为 $2^{i+1}$ 的空闲分区；随后，继续检查新生成的 $2^{i+1}$ 分区是否存在同大小的伙伴，若有，则**再次合并**，以此类推。因此，伙伴算法在一次回收过程中可能触发多次合并，但每次合并仅发生在两个大小相等且互为伙伴的空闲分区之间。

# 2024–28



操作系统 · 单项选择题 · 2 分

若进程P中的线程T先打开文件，得到文件描述符fd，再创建两个线程Ta和Tb，则下列资源中，Ta与Tb可共享的是

I. 进程P的地址空间

II. 线程T的栈

III. 文件描述符fd

A. 仅I

B. 仅I、III

C. 仅II、III

D. I、II、III

## 王道知识与解析 · 原解析已定位

【对应知识】王道2027操作系统 · 原PDF第58页
王道2027操作系统：线程属性、私有状态与共享资源

考点追踪 ▶▶ 线程的特点（2019、2011、2024）

线程可被通俗地理解为“轻量级进程”，它是**程序执行流的最小单元**，也是**处理器调度的基本单位**。在多线程操作系统中，进程不再作为基本的执行实体，但仍保留与执行相关的状态。所谓进程处于“执行”状态，实际上是指该进程中的某个线程正在运行。线程的主要属性如下。

（1）轻量级执行实体

线程本身不拥有独立的系统资源（如地址空间、打开的文件等），但拥有运行所必需的私有数据结构，包括**线程 ID**、**程序计数器**、**寄存器集合**和**栈**。每个线程都具有唯一的标识符和一个线程控制块（TCB），用于记录其执行上下文（如寄存器状态、栈指针等）。

（2）共享代码段

同一进程中的多个线程可并发执行相同的程序代码。例如，当多个用户同时访问一个 Web 服务器时，服务器进程可创建多个线程，各自服务不同的用户请求，显著提升并发能力。

（3）资源共享

同一进程内的所有线程共享该进程的代码段、数据段、堆、文件描述符等系统资源。这种机制使得线程间的通信和数据共享非常高效，无须复杂的进程间通信（IPC）机制。

（4）独立调度单位

线程是 CPU 调度和执行的基本单位。在单 CPU 系统中，多个线程通过时间片轮转等方式轮流使用 CPU；在多 CPU 系统中，不同线程可并行运行于多个 CPU 核心上。若多个核心同时执行

【对应知识】王道2027操作系统 · 原PDF第58页（续）
王道2027操作系统：线程属性、私有状态与共享资源

同一进程的不同线程，则能显著缩短该进程的整体处理时间。

【原解析 + 知识】王道2027操作系统 · 原PDF第78页
王道2027操作系统 §2.1 选择题73原解析

**73. B**

线程可理解为轻量级进程，仅拥有一点必不可少、能保证独立运行的资源。例如，在每个线程中都有一个用于控制线程运行的线程控制块（TCB）、用于指示被执行指令序列的程序计数器、保留局部变量、少数状态参数和返回地址等的一组寄存器和堆栈。因此，线程 T 的栈空间是线程 T 所独有的，不会被线程 Ta 和 Tb 共享。多个线程共享所属进程所拥有的资源，进程 P 的线程可以访问进程 P 的全部地址空间和资源（如已打开的文件、定时器、信号量机构等）以及它申请到的 I/O 设备等。综上所述，Ta 和 Tb 可共享的是进程 P 的地址空间和文件描述符 fd。

# 2024–29



操作系统　单项选择题 · 2 分

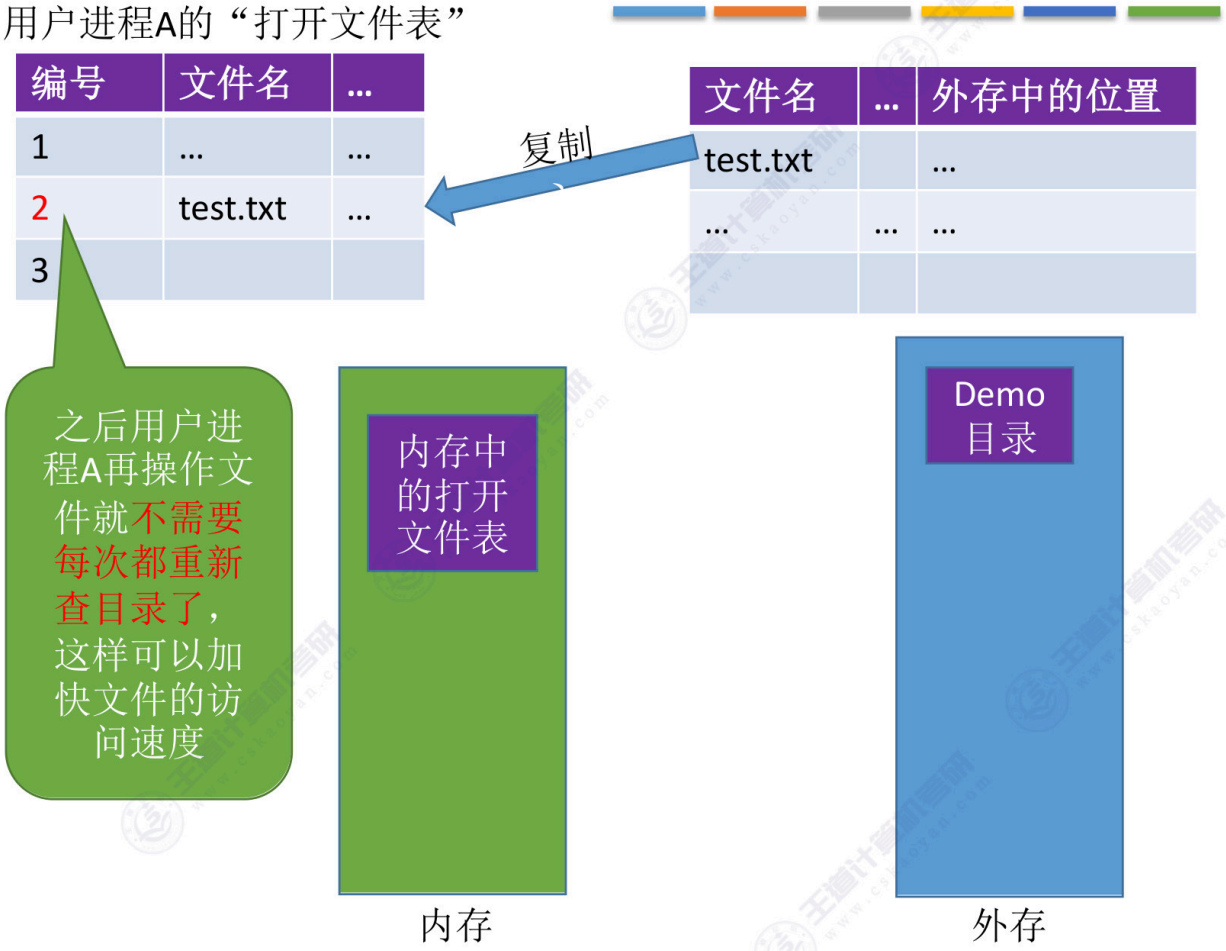
下列系统调用的实现中，包含文件按名查找功能的是

A. open()
B. read()
C. write()
D. close()

王道知识与解析 · 原解析已定位

【对应知识】4.1_7_文件的基本操作 · 原PDF第6页
王道原课件：4.1_7_文件的基本操作

# 打开文件




在很多操作系统中，在对文件进行操作之前，要求用户先使用 open 系统调用“打开文件”，需要提供的几个主要参数：

1. 文件存放路径（“D:/Demo”）
2. 文件名（“test.txt”）
3. 要对文件的操作类型（如：r 只读；rw 读写等）

操作系统在处理 open 系统调用时，主要做了几件事：

1. 根据文件存放路径找到相应的目录文件，从目录中找到文件名对应的的目录项，并检查该用户是否有指定的操作权限。
2. 将目录项复制到内存中的“打开文件表”中。并将对应表目的编号返回给用户。之后用户使用打开文件表的编号来指明要操作的文件。

王道考研/CSKAOYAN.COM

对应原题本第124页；点击此行返回题目。

【原解析 + 知识】王道2027操作系统 · 原PDF第300页
王道2027操作系统 §4.2 选择题68原解析

**68. A**

当用户首次对某文件发出操作请求时，需要先用系统调用 open()将该文件打开。所谓“打开”，是指系统利用文件名检索到指定文件的目录项后，将文件的属性（包括该文件在外存中的物理位

【原解析 + 知识】王道2027操作系统 · 原PDF第301页
王道2027操作系统 §4.2 选择题68原解析

置）从外存复制到内存的打开文件表的一个表目中，并给用户返回一个文件描述符 fd。此后（只要文件未被关闭），用户对该文件的所有操作就都是通过文件描述符 fd 来进行的。

对应原题本第124页；点击此行返回题目。

# 2024-30



操作系统　单项选择题 · 2 分

假设某系统使用时间片轮转调度算法进行CPU调度，时间片大小为5 ms，系统共有10个进程，初始时均处于就绪队列，执行结束前仅处于执行态或就绪态。若队尾的进程P所需CPU时间最短，时间为25 ms，在不考虑系统开销的情况下，则进程P的周转时间为

A. 200 ms
B. 205 ms
C. 250 ms
D. 295 ms

王道知识与解析 · 原解析已定位

【对应知识】2.2.5_2 调度算法：时间片轮转、优先级、多级反馈队列 · 原PDF第11页
王道原课件：2.2.5_2 调度算法：时间片轮转、优先级、多级反馈队列

# 时间片轮转（RR, Round-Robin）

优先级调度

| | |
|---|---|
| 算法思想 | 公平地、轮流地为各个进程服务，让每个进程在一定时间间隔内都可以得到响应 |
| 算法规则 | 按照各进程到达就绪队列的顺序，轮流让各个进程执行一个时间片（如 100ms）。若进程未在一个时间片内执行完，则剥夺处理机，将进程重新放到就绪队列队尾重新排队。 |
| 用于作业/进程调度 | 用于进程调度（只有作业放入内存建立了相应的进程后，才能被分配处理机时间片） |
| 是否可抢占？ | 若进程未能在时间片内运行完，将被强行剥夺处理机使用权，因此时间片轮转调度算法属于抢占式的算法。由时钟装置发出时钟中断来通知CPU时间片已到 |
| 优缺点 | 优点：公平；响应快，适用于分时操作系统；<br>缺点：由于高频率的进程切换，因此有一定开销；不区分任务的紧急程度。 |
| 是否会导致饥饿 | 不会 |
| 补充 | 时间片太大或太小分别有什么影响？ |

王道考研/CSKAOYAN.COM

对应原题本第125页；点击此行返回题目。

【原解析 + 知识】王道2027操作系统 · 原PDF第105页
王道2027操作系统 §2.2 选择题54原解析

**54．C**

系统使用时间片轮转调度算法，最初共有 10 个进程处于就绪队列，进程 P 处于队尾，时间片大小为 5ms。假设当 $t=0$ 时开始进程调度，就绪队列的前 9 个进程依次运行一个时间片，则当 $t=45$ms 时调度进程 P 运行，运行 5ms 后，当 $t=50$ms 时又开始新一轮循环。进程 P 所需的 CPU 时间为 25ms，共需 5 轮循环，其他进程所需的 CPU 时间都比进程 P 的长，这 5 轮循环中其余 9 个进程都不会运行结束，因此进程 P 的周转时间为 $50\times5=250$ms。

对应原题本第125页；点击此行返回题目。

# 2024-31



操作系统 · 单项选择题 · 2 分

键盘中断服务例程执行结束时，所输入数据的存放位置是

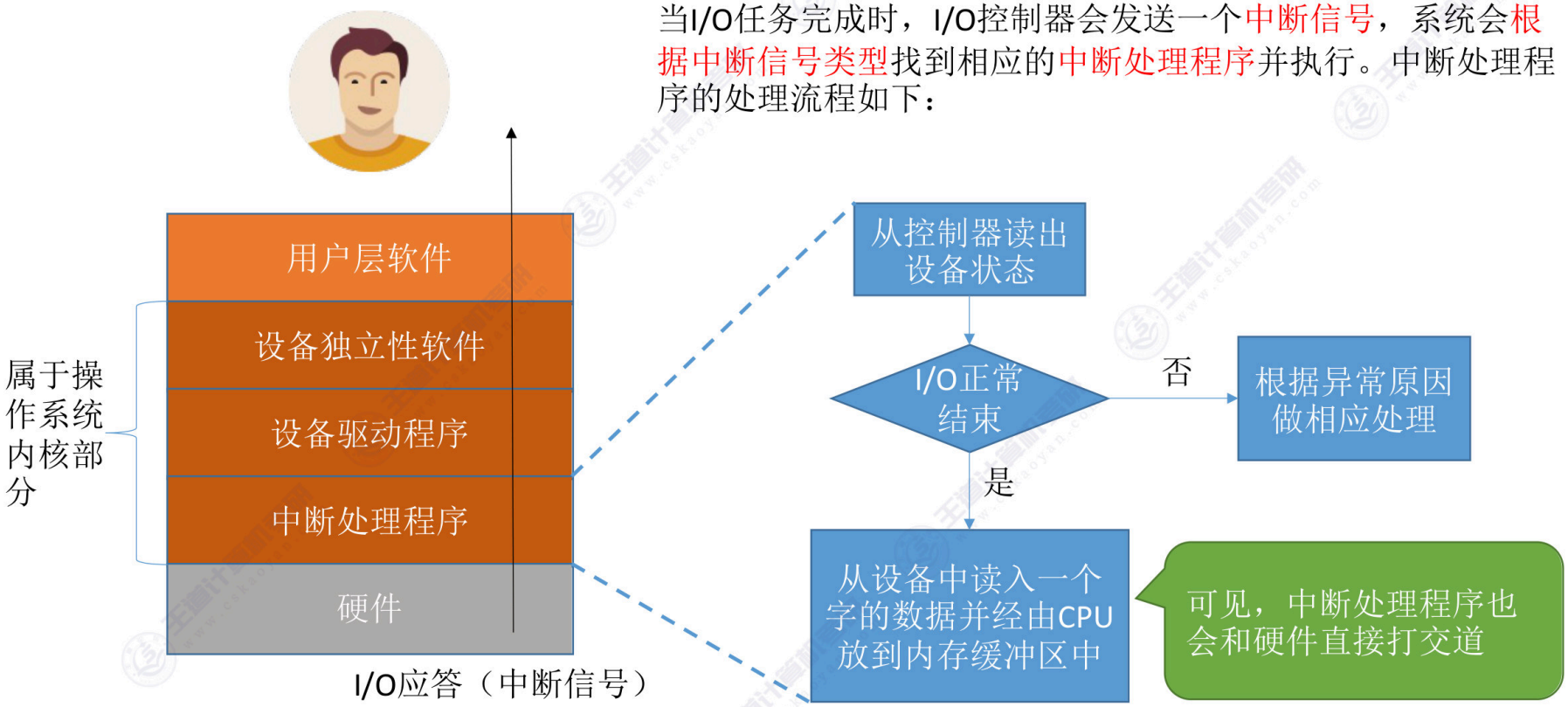
A. 用户缓冲区
B. CPU中的通用寄存器
C. 内核缓冲区
D. 键盘控制器的数据寄存器

王道知识与解析 · 原解析已定位

【对应知识】5.1_4_IO软件层次结构 · 原PDF第17页
王道原课件：5.1_4_IO软件层次结构

# 中断处理程序

当I/O任务完成时，I/O控制器会发送一个中断信号，系统会根据中断信号类型找到相应的中断处理程序并执行。中断处理程序的处理流程如下：




王道考研/CSKAOYAN.COM

对应原题本第126页；点击此行返回题目。

【原解析 + 知识】王道2027操作系统 · 原PDF第350页
王道2027操作系统 §5.2 选择题46原解析

**46．C**

以 C 语言中 scanf()函数的执行过程为例，用户请求通过键盘输入数据，当程序执行到 scanf()

【原解析 + 知识】王道2027操作系统 · 原PDF第351页
王道2027操作系统 §5.2 选择题46原解析

时，触发系统调用，CPU 切换到内核态，执行系统调用服务例程，系统调用的内核程序进行一些初始化操作，启动外设，同时阻塞该用户进程，直到键盘输入数据，键盘中断服务例程（实际上是驱动程序）将数据从 I/O 接口（键盘控制器）中的数据寄存器取出并送至内核缓冲区，然后唤醒用户进程，用户进程被唤醒后进入就绪队列，等得到 CPU 运行后，再将内核缓冲区中的数据送至用户缓冲区，最后进行系统调用的返回。综上所述，键盘中断服务例程执行结束后，所输入数据的存放位置是内核缓冲区，当系统调用结束时，所输入数据才被送至用户缓冲区。

对应原题本第126页；点击此行返回题目。

# 2024–32



操作系统 单项选择题 · 2 分

某磁盘的磁道数为 400（磁道号为 0～399），采用循环扫描算法（CSCAN）进行磁盘调度，完成对 200 号磁道的请求后，磁头向磁道号减小的方向移动。若还有 7 个磁盘请求，对应的磁道号分别为 300，120，110，0，160，210，399，则完成上述磁盘访问请求后磁头移动的距离是

A. 599
B. 619
C. 788
D. 799

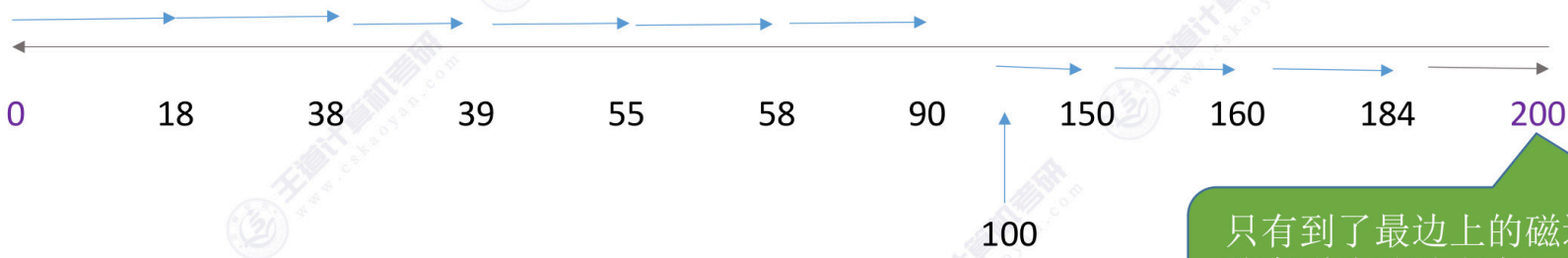
王道知识与解析 · 原解析已定位

【对应知识】5.3_2_磁盘调度算法 · 原PDF第12页
王道原课件：5.3_2_磁盘调度算法

# 循环扫描算法（C-SCAN）

SCAN算法对于各个位置磁道的响应频率不平均，而 C-SCAN 算法就是为了解决这个问题。规定只有磁头朝某个特定方向移动时才处理磁道访问请求，而返回时直接快速移动至起始端而不处理任何请求。

假设某磁盘的磁道为 0~200号，磁头的初始位置是100号磁道，且此时磁头正在往磁道号增大的方向移动，有多个进程先后陆续地请求访问 55、58、39、18、90、160、150、38、184 号磁道




磁头总共移动了 (200-100) + (200-0) + (90-0)= 390 个磁道
响应一个请求平均需要移动 390/9 = 43.3 个磁道（平均寻找长度）
优点：比起SCAN 来，对于各个位置磁道的响应频率很平均。
缺点：只有到达最边上的磁道时才能改变磁头移动方向，事实上，处理了184号磁道的访问请求之后就不需要再往右移动磁头了；并且，磁头返回时其实只需要返回到18号磁道即可，不需要返回到最边缘的磁道。另外，比起SCAN算法来，平均寻道时间更长。

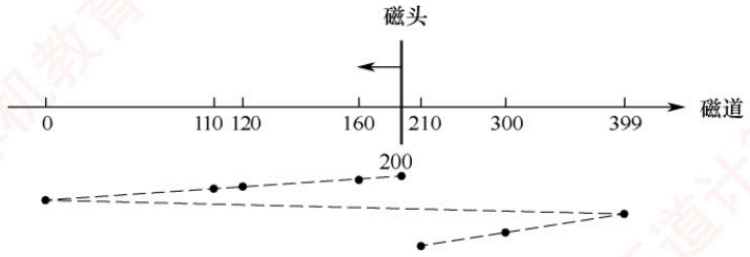
王道考研/CSKAOYAN.COM

对应原题本第127页；点击此行返回题目。

【原解析 + 知识】王道2027操作系统 · 原PDF第368页
王道2027操作系统 §5.3 选择题34原解析

**34．C**

采用 C-SCAN 算法进行磁盘调度，完成 200 号磁道的请求后，磁头向磁道号减小的方向移动，依次处理 160, 120, 110, 0 号磁道的请求；然后，磁头向磁道号增大的方向快速移动至 399 号磁道，在快速移动的过程中不处理任何请求；完成 399 号磁道的请求后，依次处理 300, 210 号磁道的请求，所以磁头移动的总距离是$(200-0)+(399-0)+(399-210)=788$。调度过程如下图所示。

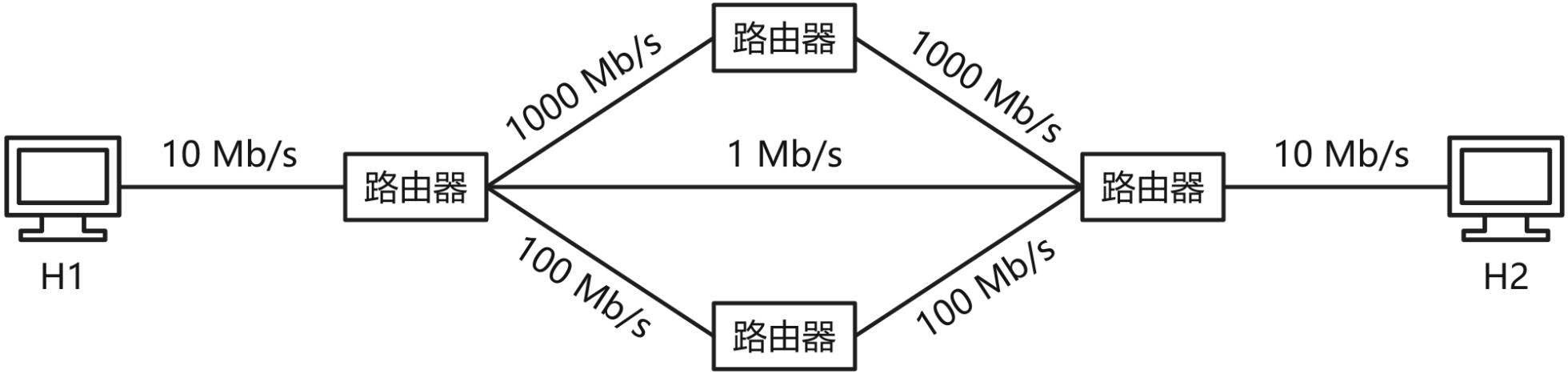


# 2024–33



计算机网络　单项选择题 · 2 分

若某分组交换网络及每段链路的带宽如下图所示，则 H1 到 H2 的最大吞吐量约为




A. 1 Mb/s
B. 10 Mb/s
C. 100 Mb/s
D. 1 000 Mb/s

王道知识与解析 · 原解析已定位

【原解析 + 知识】王道2027计算机网络 · 原PDF第26页

**16**. B

H1 到 H2 有三条路径。H1 和 H2 各自连接路由器的链路带宽均为 10Mb/s，这是所有路径共有的端侧链路。现在假设每个数据段为 10Mb，分别分析各路径的吞吐量。

**路径一**：中间两段链路带宽均为 1000Mb/s，受限于 H1、H2 直连路由器的链路带宽 10Mb/s，无论中间链路有多快，数据从 H1 发出需 1s（10Mb÷10Mb/s，受限于自身的 10Mb/s 出口），到达 H2 前最后一跳也需 1s。吞吐量约为 10Mb/s。

**路径二**：中间一段链路带宽为 1Mb/s，H1 发送一个数据段需 1s，但经过中间 1Mb/s 链路时，转发时延为 10Mb÷1Mb/s = 10s（成为瓶颈），因此 H2 每隔 10s 收到一个数据段，吞吐量约为 1Mb/s。

**路径三**：中间两段链路带宽均为 100Mb/s，与路径一的分析类似，吞吐量约为 10Mb/s。由此可见，端到端的吞吐量由路径中带宽最小的链路（瓶颈）决定。

对应原题本第128页；点击此行返回题目。

# 2024-34



计算机网络 · 单项选择题 · 2 分

在下列二进制数字调制方法中，需要 2 个不同频率载波的是

A. ASK
B. PSK
C. FSK
D. DPSK

王道知识与解析 · 原解析已定位

【原解析 + 知识】王道2027计算机网络 · 原PDF第53页

**28.** C

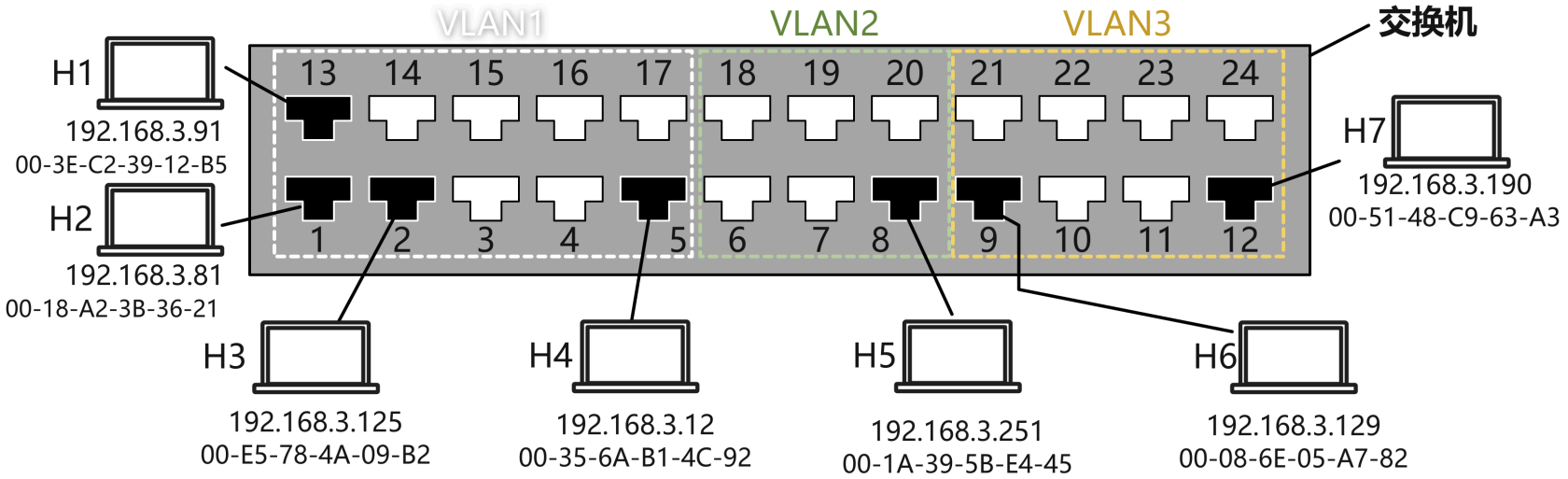
FSK（Frequency Shift Keying，频移键控）使用两个不同频率的载波分别表示 0 和 1，是唯一需要两个载波频率的二进制调制方式。ASK（Amplitude Shift Keying，幅移键控）改变幅度，PSK（Phase Shift Keying，相移键控）和 DPSK（差分相移键控）改变相位，均使用单一频率载波。

# 2024–35



计算机网络 · 单项选择题 · 2 分

如题 35 图所示的支持 VLAN 划分的交换机，已按端口划分了 3 个 VLAN，部分端口连接主机的 IP 地址和 MAC 地址如图中所示，ARP 表结构为＜IP 地址，MAC 地址，TTL＞。下列选项中，不会出现在 H4 的 ARP 表中的是




A. 192.168.3.81, 00-18-A2-3B-36-21, 14:32:00
B. 192.168.3.91, 00-3E-C2-39-12-B5, 14:37:00
C. 192.168.3.125, 00-E5-78-4A-09-B2, 14:45:00
D. 192.168.3.129, 00-08-6E-05-A7-82, 14:52:00

王道知识与解析 · 原解析已定位

【原解析 + 知识】王道2027计算机网络 · 原PDF第186页

**71．D**

ARP 用于解决局域网上的主机或路由器的 IP 地址和 MAC 地址的映射问题。因此 H4 的 ARP 表中只有和 H4 处于同一虚拟局域网上的主机的 IP 地址和 MAC 地址的映射，H4 位于 VLAN1，H6 位于 VLAN3，因此 D 表示的 H6 的 IP 地址与 MAC 地址的映射不应出现在 H4 的 ARP 表中。

对应原题本第130页；点击此行返回题目。

# 2024–36



计算机网络 · 单项选择题 · 2 分

在采用 CSMA/CA 的 802.11 无线局域网中，DIFS = 128 μs，SIFS = 28 μs，RTS、CTS 和 ACK 帧的传输时延分别是 3 μs、2 μs 和 2 μs，忽略信号传播时延。若主机 A 欲向 AP 发送一个总长度为 1 998 B 的数据帧，无线链路带宽为 54 Mb/s，则隐藏站 B 收到 AP 发送的 CTS 帧时，设置的网络分配向量 NAV 的值是

A. 326 μs
B. 354 μs
C. 385 μs
D. 513 μs

王道知识与解析 · 原解析已定位

【原解析 + 知识】王道2027计算机网络 · 原PDF第123页

**48.** B

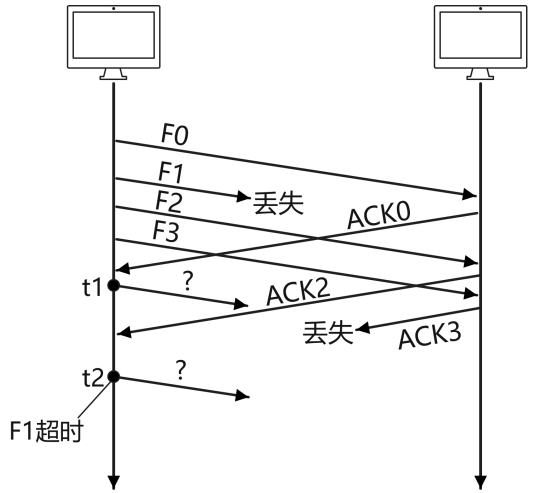
数据帧的长度为 1998B，链路带宽为 54Mb/s，因此数据帧的发送时延为 1998B÷54Mb/s = 296μs。网络分配向量（NAV）指出了信道忙的持续时间，含义是“正在通信的两个站点以外的站点都不能在这段时间内发送数据”。CSMA/CA 协议中的 RTS 帧、CTS 帧和数据帧都携带占用信道的持续时间，当 A 站广播一个 RTS 帧时，将占用信道的持续时间（SIFS + CTS + SIFS + DATA + SIFS + ACK）写入 RTS 帧的首部；当 AP 收到 RTS 帧后，广播一个 CTS 帧，将占用信道的持续时间（SIFS + DATA + SIFS + ACK）写入 CTS 帧的首部；之后传送的数据帧的首部也携带本次通信所需的持续时间。其他站收到这些帧后，根据帧中的持续时间设置自己的 NAV 值，因此隐蔽站 B 收到 AP 发送的 CTS 帧时，设置自己的 NAV 值为 SIFS + DATA + SIFS + ACK = 28μs + 296μs + 28μs + 2μs = 354μs。

# 2024–37



计算机网络 · 单项选择题 · 2 分

主机甲通过选择重传（SR）滑动窗口协议向主机乙发送帧的部分过程如题37图所示，$F_x$ 为数据帧，$ACK_x$ 为确认帧，$x$ 是位数为3比特的序号。乙只对正确接收的数据帧进行独立确认，发送窗口与接收窗口大小相同且均为最大值。甲在 $t_1$ 时刻和 $t_2$ 时刻发送的数据帧分别是



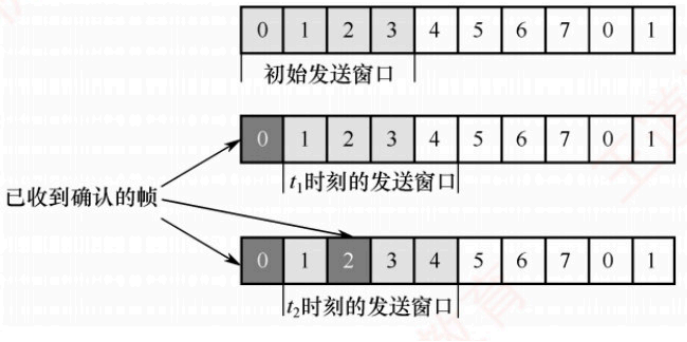

A. F1, F3
B. F1, F4
C. F3, F1
D. F4, F1

王道知识与解析 · 原解析已定位

【原解析 + 知识】王道2027计算机网络 · 原PDF第87页

**31．D**

数据帧编号的范围是 0～7，发送窗口与接收窗口大小相等且均为最大值，因此发送窗口大小 = 接收窗口大小 $=2^{3-1}=4$。甲发送 F0、F1、F2 和 F3 四个数据帧后，收到 F0 的确认，因此在 $t_1$ 时刻，发送窗口向右滑动，可以继续发送 F4，之后收到 F2 的确认，但由于 F1 丢失，甲无法收到 F1 的确认，发送窗口无法继续向右滑动，直到 $t_2$ 时刻，F1 超时，重传 F1，选项 D 正确。




对应原题本第132页；点击此行返回题目。

# 2024–38



计算机网络 · 单项选择题 · 2 分

假设主机 H 通过 TCP 向服务器发送长度为 3 000 B 的报文，往返时间 RTT=10 ms，最长报文段寿命 MSL=30 s，最大报文段长度 MSS=1 000 B，忽略 TCP 段的传输时延，报文传输结束后 H 首先请求断开连接，则从 H 请求建立 TCP 连接时刻起，到 H 进入 CLOSED 状态为止，所需的时间至少是

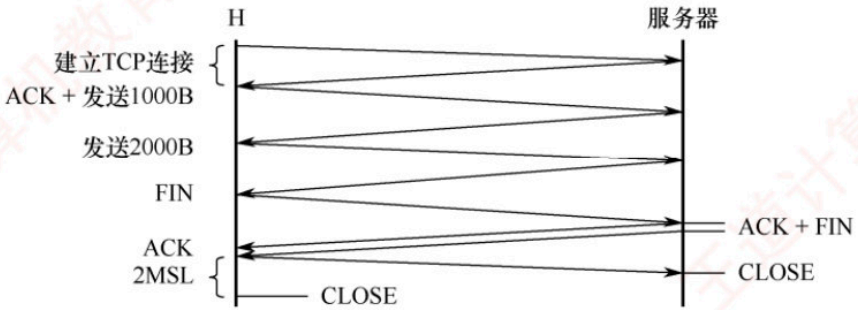
A. 30.03 s
B. 30.04 s
C. 60.03 s
D. 60.04 s

王道知识与解析 · 原解析已定位

【原解析 + 知识】王道2027计算机网络 · 原PDF第274页

**59. D**

TCP 连接建立的前两次握手耗时 1RTT。第三次握手段可携带 1000B 数据（或与数据段合并发送）。H 收到确认后，拥塞窗口增长至 2000B。因此第 3 个 RTT 内，可发送剩余 2000B 数据，至此数据传输完毕。第 4 个 RTT 开始时，H 向服务器发送 FIN 请求断开连接。题目问的是最少时间，因此服务器收到 FIN 后不再发送数据，并立即回复 ACK + FIN。第 4 个 RTT 结束时，H 收到服务器的 FIN，等待 2MSL 时间（60s）后进入 CLOSED 状态，总时间为 40ms + 60s = 60.04s。



# 2024–39



计算机网络 · 单项选择题 · 2 分

若 UDP 协议在计算校验和过程中，计算得到中间结果为 1011 1001 1011 0110 时，还需要加上最后一个 16 位数 0110 0101 1100 0101，则最终计算得到的校验和是

A. 0001 1111 0111 1011
B. 0001 1111 0111 1100
C. 1110 0000 1000 0011
D. 1110 0000 1000 0100

王道知识与解析 · 原解析已定位

【原解析 + 知识】王道2027计算机网络 · 原PDF第247页

**16. C**

UDP 检验和的计算方法是二进制反码求和再取反，二进制反码求和的运算规则：① 从低位到高位逐列进行加法运算，0 和 0 相加得 0，0 和 1 相加得 1，1 和 1 相加得 0 并产生进位 1；② 若最高位相加后产生进位，则将该进位加到结果的最低位，该过程也称回卷。计算过程如下：

```
   1011 1001 1011 0110
+  0110 0101 1100 0101
-------------------------
[1] 0001 1111 0111 1011   ——→ 最高位产生了进位，最低位要加1
+                    1
-------------------------
    0001 1111 0111 1100   ——→ 最后一步要取反
    1110 0000 1000 0011
```

# 2024-40



计算机网络 · 单项选择题 · 2 分

若浏览器不支持并行TCP连接，使用非持久的HTTP/1.0协议请求浏览1个Web页，该页中引用同一网站上7个小图像文件，则从浏览器为传输Web页请求建立TCP连接开始，到接收完所有内容为止，所需要的往返时间RTT数至少是

A. 4
B. 9
C. 14
D. 16

王道知识与解析 · 原解析已定位

【原解析 + 知识】王道2027计算机网络 · 原PDF第313页

**19. D**

浏览器不支持并行 TCP 连接，使用非持续的 HTTP/1.0 协议，因此每传输一个 Web 页和小图像文件都要建立一次 TCP 连接。第一次建立 TCP 连接时，前两次握手花 1RTT，第三次握手报文段中可以携带 HTTP 请求，服务器收到请求后返回 Web 页，共花 2RTT。之后，每传输一个图像文件都要花 2RTT。因此，到接收完所有内容，需要的总时间至少是 2×8 = 16RTT。注意，若浏览器支持并行 TCP 连接，则请求 Web 页仍要花 2RTT，但收到 Web 页后，可建立 7 个并行的 TCP 连接请求图像文件，传输图像的过程仅花 2RTT，总时间为 4RTT。

# 2024-41



数据结构 · 综合应用题 · 13 分

2023年10月26日，神舟十七号载人飞船发射取得圆满成功，再次彰显了中国航天事业的辉煌成就。载人航天工程是包含众多子工程的复杂系统工程，为了保证工程的有序开展，需要明确各子工程的前导子工程，以协调各子工程的实施。该问题可以简化、抽象为有向图的拓扑序列问题。已知有向图G采用邻接矩阵存储，类型定义如下：

```
typedef struct //图的类型定义
{
  int numVertices,numEdges;  //图的顶点数和有向边数
  char VerticesList[MAXV];  //顶点表，MAXV为已定义常量
  int Edge[MAXV][MAXV];  //邻接矩阵
}MGraph;
```

请设计算法：int uniquely(MGraph G)，判定G是否存在唯一的拓扑序列，若是，则返回1，否则返回0。要求如下。

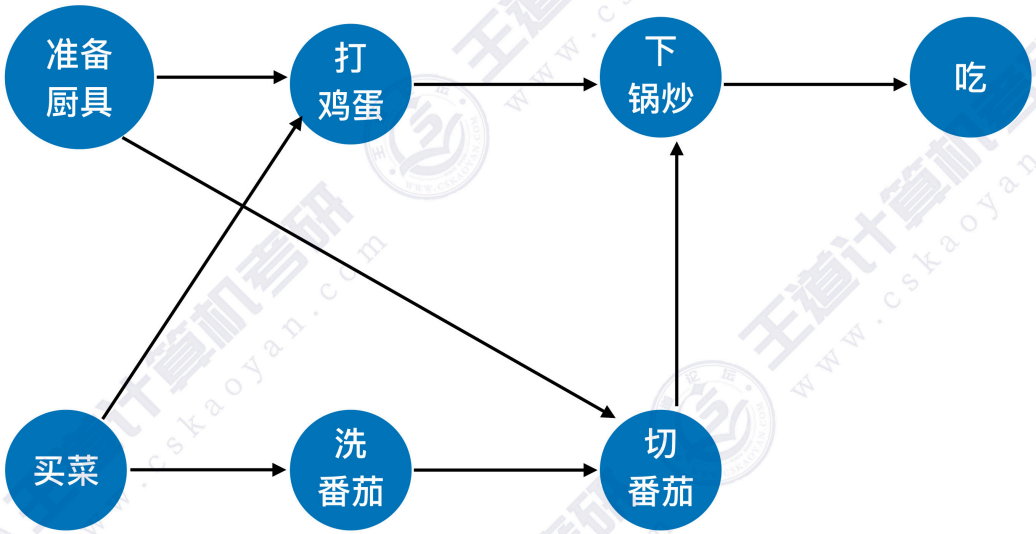
(1) 给出算法的基本设计思想。（4分）

(2) 根据设计思想，采用C或C++语言描述算法，关键之处给出注释。（9分）

王道知识与解析 · 原解析已定位

【对应知识】6.4.4 拓扑排序 · 原PDF第4页
原课件 6.4.4 拓扑排序 · PDF p4

# 拓扑排序




**拓扑排序**：在图论中，由一个**有向无环图**的顶点组成的序列，当且仅当满足下列条件时，称为该图的一个拓扑排序：
① 每个顶点出现且只出现一次。
② 若顶点$A$在序列中排在顶点$B$的前面，则在图中不存在从顶点$B$到顶点$A$的路径。

或定义为：拓扑排序是对有向无环图的顶点的一种排序，它使得若存在一条从顶点$A$到顶点$B$的路径，则在排序中顶点$B$出现在顶点$A$的后面。每个AOV网都有一个或多个拓扑排序序列。

王道考研/CSKAOYAN.COM

对应原题本第136页；点击此行返回题目。

【原解析】王道2027数据结构 · 原PDF第272页
王道2027数据结构 §6.4 综合题13原解析

**13.** 【解答】

1）算法的基本设计思想：建立图 $G$ 各顶点的入度表 degree[]。选择入度为 0 的顶点 $v$，将 $v$ 的所有邻接点的入度减 1，重复执行这个过程。若每次选中的入度为 0 的顶点有且仅有一个，且共进行了 G.numVertices 次，则意味着存在唯一的拓扑序列，返回 1，否则不存在拓扑序列，或者存在多个拓扑序列，返回 0。

2）用 C 语言描述的算法如下：

```
int uniquely(MGraph G){//判定有向图是否存在唯一的拓扑序列
    int *degree,i,j,count=0,in0=-1,prev_in0;
    degree=(int*)malloc(G.numVertices*sizeof(int));
    for(j=0;j<G.numVertices;j++){//计算各顶点的入度
        degree[j]=0;
        for(i=0;i<G.numVertices;i++)
            degree[j]+=G.Edge[i][j];
        if(degree[j]==0){
```

【原解析】王道2027数据结构 · 原PDF第273页
王道2027数据结构 §6.4 综合题13原解析

```
            if(in0==-1) in0=j;          //入度为 0 的顶点
            else in0=-2;                //有多个入度为 0 的顶点
        }
    }
    while(in0>=0){
        count++;
        prev_in0=in0;
        in0=-1;
        for(j=0;j<G.numVertices;j++)
            if(G.Edge[prev_in0][j]>0){
                if(--degree[j]==0){     //邻接点入度值减 1
                    if(in0==-1) in0=j; //入度为 0 的顶点
                    else in0=-2;        //有多个入度为 0 的顶点
                }
            }
    }
    free(degree);
    if(count==G.numVertices) return 1;
    else return 0;
}
```

对应原题本第136页；点击此行返回题目。

# 2024–42



数据结构 综合应用题 · 10 分

将关键字序列20，3，11，18，9，14，7依次存储到初始为空、长度为11的散列表HT中，散列函数H(key)=(key×3)%11。H(key)计算出的初始散列地址为$H_0$，发生冲突时探查地址序列是$H_1$，$H_2$，$H_3$，…，其中，$H_k=(H_0+k^2)\%11$，$k=1,2,3,\ldots$。

请回答下列问题。

(1) 画出所构造的HT，并计算HT的装填因子。（6分）

(2) 给出在HT中查找关键字14的关键字比较序列。（2分）

(3) 在HT中查找关键字8，确认查找失败时的散列地址是多少？（2分）

王道知识与解析 · 原解析已定位

【对应知识】7.5.4 散列查找的性能分析 · 原PDF第6页
原课件 7.5.4 散列查找的性能分析 · PDF p6

# 装填因子

**散列表的装填因子** $\alpha = \dfrac{\text{表中记录数}n}{\text{散列表长度}m}$

反映了一个散列表“满”的程度

**显然，装填因子越大，越容易发生冲突。从而导致插入、查找操作效率降低，ASL增大**

例：如上所示，散列表HT长度为16，散列函数 H(key)=key%13，其中已插入12个元素，则装填因子 α = 12/16 = 0.75

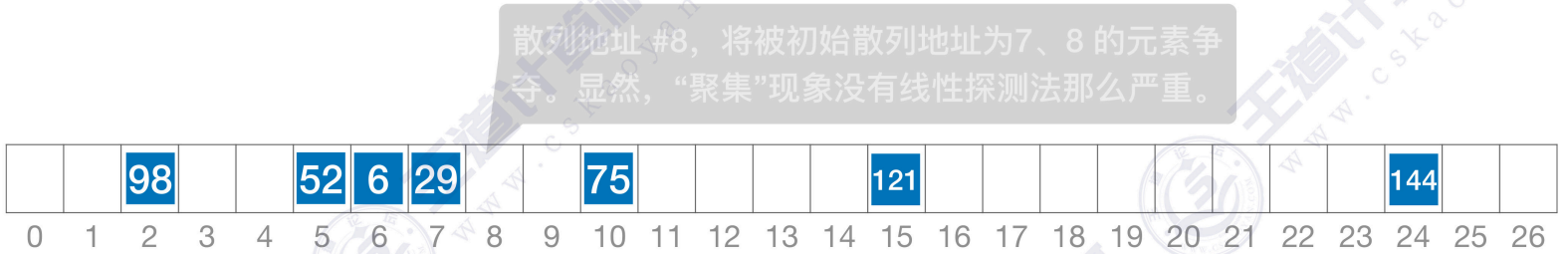
王道考研/CSKAOYAN.COM

对应原题本第137页；点击此行返回题目。

【对应知识】7.5.4 散列查找的性能分析 · 原PDF第11页
原课件 7.5.4 散列查找的性能分析 · PDF p11

# 使用“平方探测法”减少聚集现象

例：散列表长度为27，散列函数 H(key)=key%23，采用平方探测法（二次探测法）处理冲突




**平方探测法（又称二次探测法）：$d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \ldots, k^2, -k^2$，其中$k \le m/2$**

$d_0 = 0$，$d_1 = 1$，$d_2 = -1$，$d_3 = 4$，$d_4 = -4$，$d_5 = 9$，$d_6 = -9$….

**结论：采用线性探测法处理冲突更容易导致“聚集（堆积）现象”，用其他方法处理冲突可以将元素“打散”，从而减少“聚集（堆积）现象”**

王道考研/CSKAOYAN.COM

对应原题本第137页；点击此行返回题目。

【原解析】王道2027数据结构 · 原PDF第341页
王道2027数据结构 §7.5 综合题7原解析

**07. 【解答】**

1）① $H(20)=5$，装入地址 5；② $H(3)=9$，装入地址 9；③ $H(11)=0$，装入地址 0；④ $H(18)=10$，装入地址 10；⑤ $H(9)=5$，冲突，$H_1(9)=(5+1)\%11=6$，装入地址 6；⑥ $H(14)=9$，冲突，$H_1(14)=(9+1)\%11=10$，冲突，$H_2(14)=(9+4)\%11=2$，装入地址 2。散列表如下。

【原解析】王道2027数据结构 · 原PDF第342页
王道2027数据结构 §7.5 综合题7原解析

| 散列地址 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 关键字 | 11 | | 14 | 7 | | 20 | 9 | | | 3 | 18 |

装填因子 $\alpha=7/11$。

2）$H(14)=9$，和关键字 3 比较，不命中；$H_1(14)=(9+1)\%11=10$，和 18 比较，不命中；$H_2(14)=(9+4)\%11=2$，和 14 比较，命中。因此，关键字比较序列是 3, 18, 14。

3）$H(8)=2$，和关键字 14 比较，不命中；$H_1(8)=(2+1)\%11=3$，和 7 比较，不命中；$H_2(8)=(2+4)\%11=6$，和 9 比较，不命中；$H_3(8)=(2+9)\%11=0$，和 11 比较，不命中；$H_4(8)=(2+16)\%11=7$，是空位置，确认查找失败。因此，确认查找失败时的散列地址是 7。

对应原题本第137页；点击此行返回题目。

# 2024–43

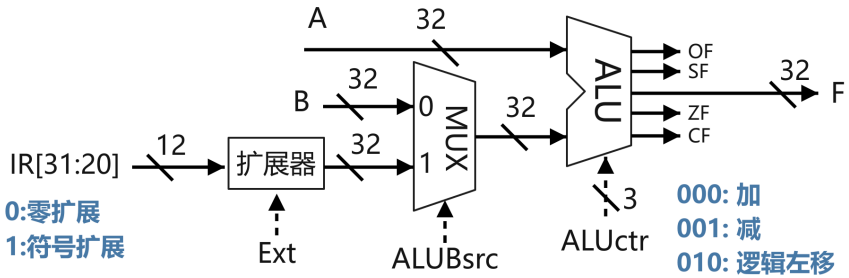


组成原理　综合应用题 · 13 分

假定计算机M字长为32位，按字节编址，采用32位定长指令字，指令add、slli和lw的格式、编码和功能说明如题43图(a)所示。

| 指令 | 31–25 | 24–20 | 19–15 | 14–12 | 11–7 | 6–0 | |
|---|---|---|---|---|---|---|---|
| add | 0000000 | rs2 | rs1 | 000 | rd | 0110011 | R[rd] <-R[rs1]+R[rs2] |
| slli | 0000000 | shamt | rs1 | 010 | rd | 0010011 | R[rd] <-R[rs1]<<shamt |
| lw | imm | | rs1 | 010 | rd | 0000011 | R[rd] <-M[R[rs1]+imm] |

其中，R[x] 表示通用寄存器 x 的内容，M[x] 表示地址为 x 的存储单元内容，shamt 为移位位数，imm 为补码表示的偏移量。
题43图（b）给出了计算机 M 的部分数据通路及其控制信号（用带箭头虚线表示），其中，A 和 B 分别表示从通用寄存器 rs1 和 rs2 中读出的内容；IR[31:20] 表示指令寄存器中的高 12 位；控制信号 Ext 为 0、1 时扩展器分别实现零扩展、符号扩展，ALUctr 为 000、001、010 时 ALU 分别实现加、减、逻辑左移运算。




请回答下列问题。
（1）计算机 M 最多有多少个通用寄存器？为什么 shamt 字段占 5 位？（2 分）
（2）执行 add 指令时，控制信号 ALUBsrc 的取值应是什么？若 rs1 和 rs2 寄存器内容分别是 8765 4321H 和 9876 5432H，则 add 指令执行后，ALU 输出端 F、OF 和 CF 的结果分别是什么？若该 add 指令处理的是无符号整数，则应根据哪个标志判断是否溢出？（5 分）
(3) 执行 slli 指令时，控制信号 Ext 的取值可以是 0 也可以是 1，为什么？(2分)
(4) 执行 lw 指令时，控制信号 Ext、ALUctr 的取值分别是什么？(2分)
(5) 若一条指令的机器码是 A040 A103H，则该指令一定是 lw 指令，为什么？若执行该指令时，R[01H]=FFFF A2D0H，则所读取数据的存储地址是什么？(2分)

【原解析 + 知识】王道2027计算机组成原理 · 原PDF第201页
王道教材 §4.3 综合题05 完整原解析

**05.** 【解答】

1）最多有 $2^5 = 32$ 个通用寄存器。M 字长为 32 位，因此通用寄存器宽度为 32 位，于是 shamt 字段占 $\log_2 32 = 5$ 位。

2）add 指令的两个源操作数均来自通用寄存器，因此控制信号 ALUBsrc = 0。rs1 = 8765 4321H，rs2 = 9876 5432H，则 rs1 + rs2 = 8765 4321H + 9876 5432H = 1FDB 9753H，在计算过程中，次高位向最高位的进位为 0，最高位产生的进位为 1，因此 OF = 0⊕1 = 1。add

【原解析 + 知识】王道2027计算机组成原理 · 原PDF第202页
王道教材 §4.3 综合题05 完整原解析

做的是加法操作，sub = 0，因此 CF = 0⊕1 = 1。无符号数根据 CF 判断是否溢出。

3）因为 slli 指令的移位位数只使用 IR[31:20]中的低 5 位，与高位 IR[31:25]及扩展出来的位无关，所以 Ext 取值可以是 0，也可以是 1。

4）lw 指令的功能是将主存地址为 R[rs1] + imm 的数据加载到目标寄存器中，需要首先通过 ALU 计算访存有效地址，imm 是补码表示的 12 位有符号数，在和 R[rs1]中的 32 位数相加时，需要进行符号扩展，R[rs1]中的数和 imm 符号扩展后的数做的是加法。因此，Ext = 1；ALUctr = 000。

5）因为 A040 A103H = 101000000100 00001 010 00010 0000011B，根据指令格式中 IR[6:0] = 0000011B，IR[14:12] = 010B，可以判定该指令是 lw 指令。lw 指令的高 12 位 = A04H，经过符号扩展后，得到 32 位机器码为 FFFF FA04H，所读取数据的存储地址为 FFFF A2D0H + FFFF FA04H = FFFF 9CD4H。

对应原题本第138页；点击此行返回题目。

# 2024-44



组成原理　综合应用题 · 10 分

**补充条件（来源：2024 年第 43 题）**

假定计算机M字长为32位，按字节编址，采用32位定长指令字，指令add、slli和lw的格式、编码和功能说明如题43图(a)所示。

| 指令 | 31 25 | 24 20 | 19 15 | 14 12 | 11 7 | 6 0 | |
|---|---|---|---|---|---|---|---|
| add | 0000000 | rs2 | rs1 | 000 | rd | 0110011 | R[rd] <-R[rs1]+R[rs2] |
| slli | 0000000 | shamt | rs1 | 010 | rd | 0010011 | R[rd] <-R[rs1]<<shamt |
| lw | imm | | rs1 | 010 | rd | 0000011 | R[rd] <-M[R[rs1]+imm] |

其中，R[x] 表示通用寄存器 x 的内容，M[x] 表示地址为 x 的存储单元内容，shamt 为移位位数，imm 为补码表示的偏移量。

**本题题干**

对于题 43 中的计算机 M，C 语言程序 P 包含的语句 “sum += a[i];” 在 M 中对应的指令序列 S 如下。

```
slli r4, r2, 2 //R[r4] ← R[r2] << 2
add r4, r3, r4 //R[r4] ← R[r3] + R[r4]
lw r5, 0(r4) //R[r5] ← M[R[r4] + 0]
add r1, r1, r5 //R[r1] ← R[r1] + R[r5]
```

已知变量 i、sum 和数组 a 都为 int 型，通用寄存器 r1～r5 的编号为 01H～05H。

请回答下列问题。

(1) 根据指令序列 S 中每条指令的功能，写出存放数组 a 的首地址、变量 i 和 sum 的通用寄存器编号。(3 分)

(2) 已知 M 为小端方式计算机，采用页式存储管理方式，页大小为 4 KB。若执行到指令序列 S 中第 1 条指令时，i=5 且 r1 和 r3 的内容分别为 0000 1332H 和 0013 DFF0H，从地址 0013 DFF0H 开始的存储单元内容如题 44 图所示，则执行

“sum+=a[i];” 语句后，a[i]的地址、a[i]和sum的机器数分别是什么（用十六进制表示）？a[i]所在页的页号是多少？此次执行中，数组a至少存放在几页中？（5分）

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| 0013 DFF0 | FF | FF | FF | 7C | 70 | FE | FF | FF |
| 0013 DFF8 | 00 | 00 | 00 | 0C | 3C | 02 | 01 | FF |
| 0013 E000 | F0 | F1 | 00 | 00 | DC | EC | FF | FF |
| 0013 E008 | FF | FF | 01 | 02 | 00 | 00 | 01 | 02 |

(3)指令“slli r4, r2, 2”的机器码是什么（用十六进制表示）？若数组a改为short类型，则指令序列S中slli指令的汇编形式应是什么？（2分）

【原解析 + 知识】王道2027计算机组成原理 · 原PDF第202页
王道教材 §4.3 综合题06 完整原解析

**06.【解答】**

1）变量 i 存放在 r2 中。R[r4]←R[r2]<<2 的功能是将 i 左移两位，即将 i×4 的值送到 r4。数组 a 的首地址（a[0]的地址）存放在 r3 中。R[r4]←R[r3]+R[r4]的功能是计算 addr(a[0]) + i×4，即将 a[i]的地址送到 r4。R[r5]←M[R[r4]+0]的功能是将 a[i]送到 r5。变量 sum 存放在 r1 中。R[r1]←R[r1]+R[r5]的功能是将 sum + a[i]的结果送回 r1。

2）数组 a 的首地址为 0013 DFF0H，从表中可以看出，a[5]的地址为 0013 E004H。M 为小端方式计算机，根据表格可知，a[5]的机器数为 FFFF ECDCH。页大小为 4KB，因此页内偏移量为低 12 位，页号为地址高 20 位，即 0013EH。sum 的初值为 0000 1332H，执行 sum+=a[i] 后，sum 的机器数更新为 0000 1332H + FFFF ECDCH = 0000 000EH。表格中的数据包含了页号为 0013DH 和 0013EH 两个不同页面的数据，所以数组 a 至少存放在 2 页中。

3）指令机器码 = 0000000 00010 00010 010 00100 0010011 = 0021 2213H。若数组 a 改为 short 型，则每个元素占 2 字节，a[i]的地址为 addr(a[0]) + i×2，因此汇编形式是 slli r4, r2, 1。

对应原题本第139页；点击此行返回题目。

# 2024-45



操作系统　综合应用题 · 7 分

某计算机按字节编址，采用页式虚拟存储管理方式，虚拟地址和物理地址的长度均为32位，页表项的大小为4字节，页大小为4 MB，虚拟地址结构如下。

`[页号(10位) | 页内偏移量(22位)]`

进程P的页表起始虚拟地址为B8C0 0000H，被装载到从物理地址6540 0000H开始的连续主存空间中。

请回答下列问题，要求答案用十六进制表示。

（1）若 CPU 在执行进程 P 的过程中，访问虚拟地址 `1234 5678H` 时发生了缺页异常，经过缺页异常处理和 MMU 地址转换后得到的物理地址是 `BAB4 5678H`。在此次缺页异常处理过程中，需要为所缺页分配页框并更新相应的页表项，则该页表项的虚拟地址和物理地址分别是什么？该页表项中的页框号更新后的值是什么？（3分）
（2）进程 P 的页表所在页的页号是什么？该页对应的页表项的虚拟地址是什么？该页表项中的页框号是什么？（4分）

王道知识与解析 · 原解析已定位

【对应知识】3.1.3_1 基本分页存储管理的基本概念 · 原PDF第16页
王道原课件：3.1.3_1 基本分页存储管理的基本概念

# 逻辑地址结构

分页存储管理的逻辑地址结构如下所示：

| 31 ..... 12 | 11 ..... 0 |
| --- | --- |
| 页号 P | 页内偏移量 W |

地址结构包含两个部分：前一部分为页号，后一部分为页内偏移量 W。在上图所示的例子中，地址长度为 32 位，其中 0~11位 为“页内偏移量”，或称“页内地址”；12~31 位为“页号”。

如果有 K 位表示“页内偏移量”，则说明该系统中一个页面的大小是 $2^K$ 个内存单元
如果有 M 位表示“页号”，则说明在该系统中，一个进程最多允许有 $2^M$ 个页面

重要重要重要！！！
页面大小 ↔ 页内偏移量位数
➔逻辑地址结构

Tips：有些奇葩题目中页面大小有可能不是2的整数次幂，这种情况还是得用最原始的方法计算：
页号 = 逻辑地址 / 页面长度 （取除法的整数部分）
页内偏移量 = 逻辑地址 % 页面长度（取除法的余数部分）

王道考研/CSKAOYAN.COM

对应原题本第140页；点击此行返回题目。

【对应知识】3.1.3_2 基本地址变换机构 · 原PDF第5页
王道原课件：3.1.3_2 基本地址变换机构

# 基本地址变换机构

基本地址变换机构可以借助进程的页表将逻辑地址转换为物理地址。
通常会在系统中设置一个页表寄存器（PTR），存放页表在内存中的起始地址F 和页表长度M。进程未执行时，页表的始址 和 页表长度 放在进程控制块（PCB）中，当进程被调度时，操作系统内核会把它们放到页表寄存器中。

注意：页面大小是2的整数幂
设页面大小为L，逻辑地址A到物理地址E的变换过程如下：
①计算页号 P 和页内偏移量W（ 如果用十进制数手算，则 P=A/L，W=A%L；但是在计算机实际运行时，逻辑地址结构是固定不变的，因此计算机硬件可以更快地得到二进制表示的页号、页内偏移量）
②比较页号P 和页表长度M，若 P≥M，则产生越界中断，否则继续执行。（注意：页号是从0开始的，而页表长度至少是1，因此 P=M 时也会越界）
③页表中页号P对应的页表项地址 = 页表起始地址F + 页号P * 页表项长度，取出该页表项内容b，即为内存块号。（注意区分页表项长度、页表长度、页面大小的区别。页表长度指的是这个页表中总共有几个页表项，即总共有几个页；页表项长度指的是每个页表项占多大的存储空间；页面大小指的是一个页面占多大的存储空间）
④计算 E = b * L + W，用得到的物理地址E 去访存。（如果内存块号、页面偏移量是用二进制表示的，那么把二者拼接起来就是最终的物理地址了）

王道考研/CSKAOYAN.COM

对应原题本第140页；点击此行返回题目。

【原解析 + 知识】王道2027操作系统 · 原PDF第262页
王道2027操作系统 §3.2 综合题22原解析

**22.【解答】**

1）页表项的虚拟地址为 B8C0 0000H + 48H << 2 = B8C0 0120H。页表项的物理地址为 6540 0000H + 48H << 2 = 6540 0120H。相应页表项中的页框号为 BAB4 5678H >> 22 = 2EAH。

2）进程 P 的页表所在页的页号为 B8C0 0000H >> 22 = 2E3H。页表项的虚拟地址为 B8C0 0000H + 2E3H << 2 = B8C0 0B8CH。页表项中的页框号为 6540 0000H >> 22 = 195H。

注意，符号“<<”表示左移操作，符号“>>”表示右移操作。将二进制数左移 1 位，与“乘以 2”的效果等价。页表项中的页框号为物理地址的前 10 位可以通过右移 22 位得到。

对应原题本第140页；点击此行返回题目。

# 2024–46



操作系统　综合应用题 · 8 分

计算机系统中的进程之间往往需要相互协作以完成一个任务。在某网络系统中，缓冲区B用于存放一个数据分组，对B的操作有C1、C2和C3。C1将一个数据分组写入B中，C2从B中读出一个数据分组，C3对B中的数据分组进行修改。要求B为空时才能执行C1，B非空时才能执行C2和C3。
请回答下列问题。

（1）假设进程 P1 和 P2 均需要执行 C1，实现 C1 的代码是否为临界区？为什么？（2分）
（2）假设 B 初始为空，进程 P1 执行 C1 一次，进程 P2 执行 C2 一次。请定义尽可能少的信号量，并用 `wait()`、`signal()` 操作描述进程 P1 和 P2 之间的同步或互斥关系，说明所用信号量的作用及其初值。（3分）
（3）假设 B 初始不为空，进程 P1 和 P2 各执行 C3 一次。请定义尽可能少的信号量，并用 `wait()`、`signal()` 操作描述进程 P1 和 P2 之间的同步或互斥关系，说明所用信号量的作用及其初值。（3分）

王道知识与解析 · 原解析已定位

【对应知识】2.3.4_2 用信号量实现进程互斥、同步、前驱关系 · 原PDF第3页
王道原课件：2.3.4_2 用信号量实现进程互斥、同步、前驱关系

理解：信号量 mutex 表示“进入临界区的名额”

# 信号量机制实现进程互斥

```
/*记录型信号量的定义*/
typedef struct {
  int value;              //剩余资源数
  struct process *L;      //等待队列
} semaphore;
```

1. 分析并发进程的关键活动，划定临界区（如：对临界资源打印机的访问就应放在临界区）
2. 设置互斥信号量 mutex，初值为 1
3. 在进入区 P(mutex)——申请资源
4. 在退出区 V(mutex)——释放资源

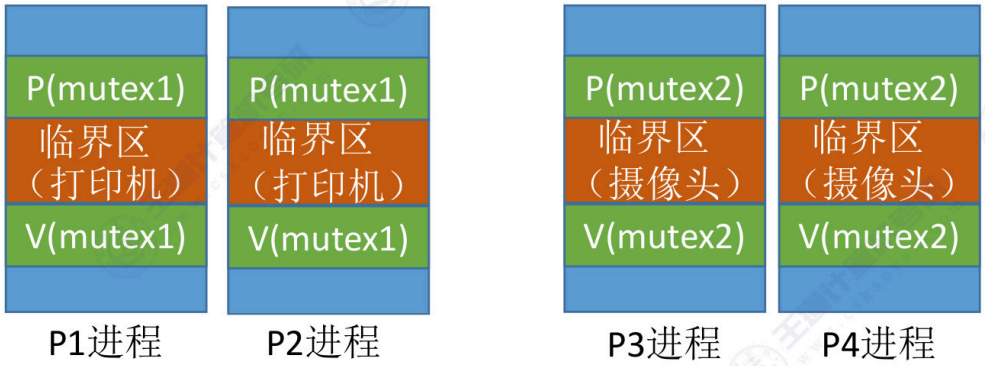
注意：对不同的临界资源需要设置不同的互斥信号量。
P、V操作必须成对出现。缺少 P(mutex) 就不能保证临界资源的互斥访问。缺少 V(mutex) 会导致资源永不被释放，等待进程永不被唤醒。

要会自己定义记录型信号量，但如果题目中没特别说明，可以把信号量的声明简写成这种形式

```
/*信号量机制实现互斥*/
semaphore mutex=1;   //初始化信号量

P1(){
  ...
  P(mutex);          //使用临界资源前需要加锁
  临界区代码段...
  V(mutex);          //使用临界资源后需要解锁
  ...
}

P2(){
  ...
  P(mutex);
  临界区代码段...
  V(mutex);
  ...
}
```




王道考研/CSKAOYAN.COM

【对应知识】2.3.4_2 用信号量实现进程互斥、同步、前驱关系 · 原PDF第5页
王道原课件：2.3.4_2 用信号量实现进程互斥、同步、前驱关系

# 信号量机制实现进程同步

用信号量实现进程同步：

1. 分析什么地方需要实现“同步关系”，即必须保证“一前一后”执行的两个操作（或两句代码）
2. 设置同步信号量 S, 初始为 0
3. 在“前操作”之后执行 V(S)
4. 在“后操作”之前执行 P(S)

技巧口诀：前V后P

```
/*信号量机制实现同步*/
semaphore S=0;   //初始化同步信号量，初始值为0
```

理解：信号量S代表“某种资源”，刚开始是没有这种资源的。P2需要使用这种资源，而又只能由P1产生这种资源

```
P1(){
   代码1;
   代码2;
   V(S);
   代码3;
}
```

释放资源

```
P2(){
   P(S);
   代码4;
   代码5;
   代码6;
}
```

保证了 代码4 一定是在 代码2 之后执行

若先执行到 V(S) 操作，则 S++ 后 S=1。之后当执行到 P(S) 操作时，由于 S=1，表示有可用资源，会执行 S--，S 的值变回 0，P2 进程不会执行 block 原语，而是继续往下执行代码4。

若先执行到 P(S) 操作，由于 S=0，S-- 后 S=-1，表示此时没有可用资源，因此P操作中会执行 block 原语，主动请求阻塞。之后当执行完代码2，继而执行 V(S) 操作， S++，使 S 变回 0，由于此时有进程在该信号量对应的阻塞队列中，因此会在 V 操作中执行 wakeup 原语，唤醒 P2 进程。这样 P2 就可以继续执行 代码4 了

王道考研/CSKAOYAN.COM

【原解析 + 知识】王道2027操作系统 · 原PDF第160页
王道2027操作系统 §2.3 综合题28原解析

**28.【解答】**

1）代码 $C_1$ 执行对 B 的写操作，且 $P_1$ 和 $P_2$ 需要互斥执行 $C_1$，因此 $C_1$ 的代码是临界区。

2）有一组互斥关系和一组同步关系。互斥关系为：$P_1$ 和 $P_2$ 需要互斥访问缓冲区 B；同步关系为：$P_1$ 执行 $C_1$ 后，$P_2$ 才能执行 $C_2$。缓冲区 B 初始为空，且最大容量为 1，因此可不用定义互斥信号量 mutex。因为在缓冲区大小为 1 的条件下，同步信号量 S 就可同时保证 $P_1$ 和 $P_2$ 互斥访问 B，符合题意“尽可能定义少的信号量”的要求。进程 $P_1$ 和 $P_2$ 的同步伪代码如下。

【原解析 + 知识】王道2027操作系统 · 原PDF第161页
王道2027操作系统 §2.3 综合题28原解析

```
Semaphore S = 0；//实现进程 P1 与 P2 的同步
```

| P1 | P2 |
|---|---|
| … | … |
| C1; | wait(S); |
| signal(S); | C2; |
| … | … |

3）缓冲区 B 初始非空，因此仅需一个互斥信号量 mutex 来保证 $P_1$ 和 $P_2$ 互斥访问 B 即可。进程 $P_1$ 和 $P_2$ 的互斥伪代码如下。

```
Semaphore mutex = 1；//实现 P1 与 P2 互斥执行 C3
```

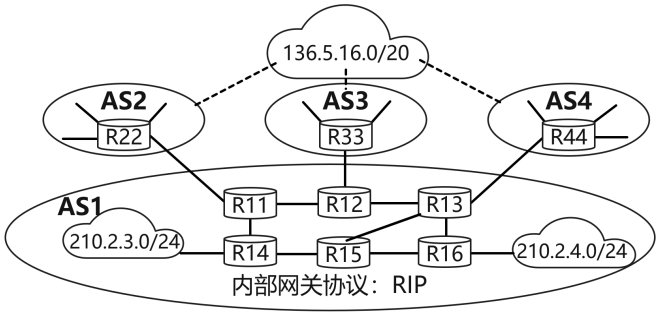
| P1 | P2 |
|---|---|
| … | … |
| wait(mutex); | wait(mutex); |
| C3; | C3; |
| signal(mutex); | signal(mutex); |
| … | … |

# 2024–47



计算机网络 · 综合应用题 · 9 分

网络空间是继陆海空天之后的“第五疆域”，网络技术是网络疆域建设与治理的基础。路由算法与协议是网络核心技术之一，对其准确认知、合理选择与应用，对于网络建设十分重要。假设现有互联网中的4个自治系统互连拓扑示意图如题47图所示。其中，AS1运行内部网关协议RIP；AS3规模较小，自治系统内任意两个主机间通信，经过路由器数量不超过15个；AS4规模较大，自治系统内任意两个主机间通信，经过路由器数量可能超过20个。




请回答下列问题。

(1) 若仅有RIP和OSPF内部网关协议供选择，则AS4应该选择哪个协议？（1分）

(2) 若AS3中的某主机向本自治系统内另一主机发送1个IP分组，为确保该IP分组能够被正常接收，则该IP分组的初始TTL值应该至少设置为多少？（1分）

(3) 假设AS1中的路由器同一时刻启动，启动后立即构建并交换初始距离向量，之后每隔30 s交换一次最新的距离向量，则从交换初始距离向量时刻算起，R11～R16路由器均获得到达网络210.2.3.0/24的正确路由，且均获得到达网络210.2.4.0/24的正确路由，至少需要多长时间？（2分）

(4) R44向R13通告到达网络136.5.16.0/20路由时，由BGP协议哪类会话完成？通过哪个BGP报文通告？R13通过BGP协议的哪类会话将该网络可达性信息通告给R14和R15？（3分）

(5) 若R14和R15均收到分别由R11、R12、R13通告的到达网络136.5.16.0/20的可达性信息如下。

目的网络：136.5.16.0/20，AS路径：AS2 AS8 AS19，下一跳：R11

目的网络：136.5.16.0/20，AS路径：AS3 AS7 AS11 AS19，下一跳：R12

目的网络：136.5.16.0/20，AS路径：AS4 AS10 AS19，下一跳：R13

则在无策略约束情况下，R14和R15更新路由表后，各自路由表中到达网络136.5.16.0/20路由的下一跳分别是什么（用路由器名称表示）？（2分）

王道知识与解析 · 原解析已定位

【原解析 + 知识】王道2027计算机网络 · 原PDF第219页

**06.【解答】**

1）RIP 限制了网络规模，其最大跳数为 15；且 RIP 路由器周期性交换完整的路由表，随着网络规模扩大，开销会显著增大。因此，AS4 应该选择 OSPF 协议。

2）IP 分组每经过一个路由器，TTL 值减 1，当 TTL 减为 0 时被丢弃。依题意，AS3 中任意两主机间通信最多经过 15 个路由器，为确保分组能被正常接收，初始 TTL 值至少应设为 16，这样在经过第 15 个路由器后，TTL 值变为 1，可成功交付。

【原解析 + 知识】王道2027计算机网络 · 原PDF第220页

3）对于网络 210.2.3.0/24：R14 直连该网络，启动后立即构建初始距离向量。首次交换（$t=0$）时，R14 将路由信息通告给 R11 和 R15，因此 R11、R14、R15 均立即获得正确的路由，如图 1 所示。经过 30s 后（第二次交换），R11 通告给 R12，R15 通告给 R13 和 R16，至此 R11～R16 均获得正确的路由，如图 2 所示，故至少需要 30s。

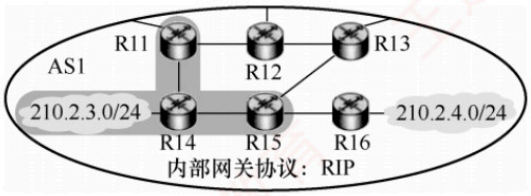



图 1

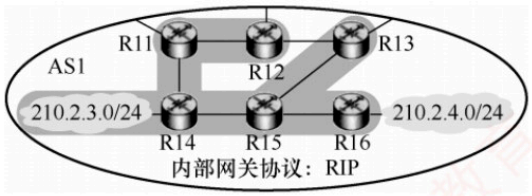



图 2

对于网络 210.2.4.0/24：R16 直连该网络。首次交换（$t=0$）时，R16 将路由信息通告给 R15，R15 获知路由，如图 3 所示。30s 后，R15 通告给 R13 和 R14，此时 R16、R15、R13、R14 全部获知路由，如图 4 所示；再经过 30s（共 60s），R14 通告给 R11，R13 通告给 R12，此时 R11～R16 全部获知路由，如图 5 所示，故至少需要 60s。

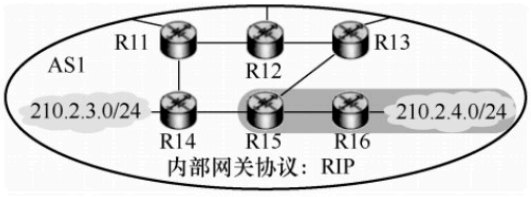



图 3

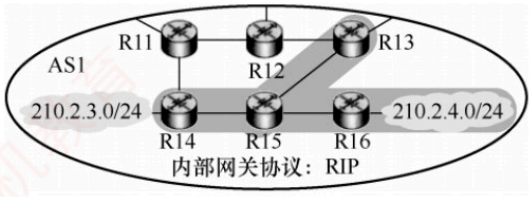



图 4

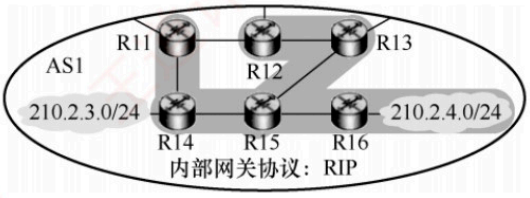



图 5

4）R44 属于 AS4，R13 属于 AS1，二者位于不同的自治系统中，因此 R44 向 R13 通告路由通过 eBGP（外部 BGP）会话完成。使用 UPDATE 报文通告路由信息。R13 向同属 AS1 的 R14 和 R15 通告该路由，通过 iBGP（内部 BGP）会话完成。

5）BGP 在无策略约束时，优先选择跳数最少的路由。依题意，从 R11 和 R13 离开都会经过 3 个 AS，从 R12 离开会经过 4 个 AS。从 R11 与 R13 离开的跳数相等。此时，应采用热

对应原题本第142页；点击此行返回题目。

【原解析 + 知识】王道2027计算机网络 · 原PDF第220页（续）

土豆路由选择算法，让分组经过最少的转发次数离开 AS1。由于 AS1 采用的是 RIP，因此 R14 会选择距离最近的 R11 转发；类似地，R15 会选择距离最近的 R13 转发。

对应原题本第142页；点击此行返回题目。